



PDF Download
3769292.pdf
16 January 2026
Total Citations: 0
Total Downloads:
1048

Latest updates: <https://dl.acm.org/doi/10.1145/3769292>

SURVEY

Machine Learning Systems: A Survey from a Data-Oriented Perspective

CHRISTIAN CABRERA, University of Cambridge, Cambridge, Cambridgeshire, U.K.

ANDREI PALEYES, University of Cambridge, Cambridge, Cambridgeshire, U.K.

PIERRE THODOROFF, University of Cambridge, Cambridge, Cambridgeshire, U.K.

NEIL D LAWRENCE, University of Cambridge, Cambridge, Cambridgeshire, U.K.

Open Access Support provided by:

University of Cambridge

Published: 20 November 2025

Online AM: 24 September 2025

Accepted: 15 September 2025

Revised: 01 September 2025

Received: 09 October 2023

[Citation in BibTeX format](#)

Machine Learning Systems: A Survey from a Data-Oriented Perspective

CHRISTIAN CABRERA, Department of Computer Science and Technology, University of Cambridge, Cambridge, United Kingdom of Great Britain and Northern Ireland

ANDREI PALEYES, Department of Computer Science and Technology, University of Cambridge, Cambridge, United Kingdom of Great Britain and Northern Ireland

PIERRE THODOROFF, Department of Computer Science and Technology, University of Cambridge, Cambridge, United Kingdom of Great Britain and Northern Ireland

NEIL LAWRENCE, Department of Computer Science and Technology, University of Cambridge, Cambridge, United Kingdom of Great Britain and Northern Ireland

Engineers are deploying ML models as parts of real-world systems with the upsurge of AI technologies. Real-world environments challenge the deployment of such systems because these environments produce large amounts of heterogeneous data, and users require increasingly efficient responses. These requirements push prevalent software architectures to the limit when deploying ML-based systems. Data-Oriented Architecture (DOA) is an emerging style that better equips systems to integrate ML models. Even though articles on deployed ML-based systems do not mention DOA, their authors make design decisions that implicitly follow DOA. Implicit decisions create a knowledge gap, limiting practitioners' ability to implement ML-based systems. This article surveys why, how, and to what extent practitioners have adopted DOA to implement ML-based systems. We overcome the knowledge gap by answering these questions and explicitly showing the design decisions and practices behind these systems. The survey follows a well-known systematic and semi-automated methodology for reviewing articles in software engineering. The majority of reviewed works partially adopt DOA. Such an adoption enables systems to address big data management, low-latency processing, resource management, security, and privacy requirements. Based on these findings, we formulate practical advice to facilitate the deployment of ML-based systems.

CCS Concepts: • **Software and its engineering**; • **Computing methodologies** → **Artificial intelligence**; **Machine learning**;

Additional Key Words and Phrases: Artificial Intelligence, Machine Learning, Real-World Deployment, Systems Architecture, Data-Oriented Architecture

This research was generously funded by the Engineering and Physical Sciences Research Council (EPSRC) and the Alan Turing Institute under grant EP/V030302/1.

Authors' Contact Information: Christian Cabrera (corresponding author), Department of Computer Science and Technology, University of Cambridge, Cambridge, United Kingdom of Great Britain and Northern Ireland; e-mail: chc79@cam.ac.uk; Andrei Paleyes, Department of Computer Science and Technology, University of Cambridge, Cambridge, United Kingdom of Great Britain and Northern Ireland; e-mail: ap2169@cam.ac.uk; Pierre Thodoroff, Department of Computer Science and Technology, University of Cambridge, Cambridge, United Kingdom of Great Britain and Northern Ireland; e-mail: pt440@cam.ac.uk; Neil Lawrence, Department of Computer Science and Technology, University of Cambridge, Cambridge, United Kingdom of Great Britain and Northern Ireland; e-mail: ndl21@cam.ac.uk.



This work is licensed under a [Creative Commons Attribution 4.0 International License](https://creativecommons.org/licenses/by/4.0/).

© 2025 Copyright held by the owner/author(s).

ACM 0360-0300/2025/11-ART115

<https://doi.org/10.1145/3769292>

ACM Reference Format:

Christian Cabrera, Andrei Paleyes, Pierre Thodoroff, and Neil Lawrence. 2025. Machine Learning Systems: A Survey from a Data-Oriented Perspective. *ACM Comput. Surv.* 58, 5, Article 115 (November 2025), 38 pages. <https://doi.org/10.1145/3769292>

1 Introduction

Machine Learning (ML) solutions have been used to address challenging problems in several domains (healthcare, science, etc.) [77, 137]. Their success is a product of the growth of available data, increasingly powerful hardware, and the development of novel ML models [41, 112]. Many of these models have demonstrated practical value, which has led to their rapid adoption in real-world software systems. The contrast between real-world environments and the more controlled environments from which ML models originate challenges the deployment of systems that integrate these ML models [27]. In particular, real-world environments produce large amounts of complex and dynamic data, which these systems must process efficiently [25, 65]. ML-based systems deployed in the real world must be scalable, autonomous, and resource-efficient, while enabling data availability, monitoring, security, and trust [80, 105, 108].

Service-oriented architectures (SOAs) and microservices are the dominant software architectural styles nowadays [11, 95]. These styles are characterised by services that separate systems' functionalities, communicate through well-defined programming interfaces (i.e., APIs), and are usually deployed in the cloud [133]. The necessity to process large volumes of data, typical for ML-based real-world systems, leads to new requirements that challenge SOA-based systems. For example, separation of concerns facilitates systems maintenance by following the divide-and-conquer principle [70, 86]. However, data monitoring and transparency requirements are arduous to satisfy because services hide the systems' data behind their APIs [34]. This situation is known as "the Data Dichotomy": while high-quality data management requires exposing systems' data, services hide it behind their interfaces [130]. Another example is that services deployed in flexible cloud platforms support highly available and scalable systems [67]. However, cloud deployments struggle to meet low-latency requirements for critical applications. The physical distance between end users and cloud data centres impacts systems' end-to-end response time [28]. Such cloud-based deployments also affect data security, privacy, and trust requirements because the ownership of the data changes from end users to cloud providers. This change of ownership also generates availability, privacy, and security concerns that can impact ML model training [4, 74].

Data-Oriented Architecture (DOA) is an emerging architectural style that complements the current practices for implementing systems. DOA considers data as the common denominator between system components [65]. Services in DOA are distributed, autonomous, and communicate with each other at the data level (i.e., data coupling) using asynchronous messages [120, 138]. DOA enables systems to achieve data availability, reusability, monitoring, and system autonomy and resource efficiency [65, 120, 138]. These properties facilitate systems integrating ML models in production environments. For example, the data coupling between components enables the storage of the states of the data that flow through it by design. A practitioner or a meta-learning system can analyse such records, identify when the data distribution changes, and update the system's learning models. Similarly, DOA enables the creation of open systems because of the autonomous communication between its components. New devices can automatically join and provide computing power to DOA-based systems. Such flexibility facilitates the deployment of ML models in constrained environments and can mitigate low-latency requirements as services are physically closer to end users [24, 132].

Even though most articles on deployed ML-based systems do not describe DOA explicitly, their authors face the same challenges that DOA aims to address. This implicit adoption creates a knowledge gap that obscures the software development practices, system design patterns, and architectural styles practitioners apply when building ML-based systems. Communities must close this gap to propose and develop novel software engineering approaches and system architectural styles that support the design, development, deployment, and decommissioning of ML-based systems, which are fundamentally different from classical software systems [23, 105]. This work contributes to closing this gap by surveying articles that report ML-based systems, explicitly showing the design decisions behind them, and analysing why, how, and to what extent their authors have implicitly adopted DOA principles. Such an analysis constitutes the first in-depth study of the DOA architectural style for ML-based systems. Based on existing literature on DOA, we formulate high-level principles of this architectural style and contextualise these principles through the ML deployment challenges. We then conduct a comprehensive survey to understand the implicit adoption of DOA principles among practitioners when implementing existing ML-based systems.¹ Based on the survey findings, this article shows the benefits and limitations of DOA, formulates practical advice for deploying ML-based systems, and discusses open challenges and research directions to advance DOA.

2 Related Work

Although DOA as an architectural style for ML-based systems is an emerging concept, the principles behind it are not new. Many domains have leveraged the benefits of applying similar principles. For instance, **data-oriented design (DOD)** applies DOA principles on a lower level of abstraction, with claims of significant improvements over analogous object-oriented programming solutions. Video game development is one domain where DOD is seeing adoption, leading to improved memory and cache utilisation [1]. For example, Coherent Labs utilised DOD while creating their game engine Hummingbird [90] to overcome the performance limitations. DOD helped developers eliminate cache misses and compiler branch mispredictions, resulting in significant performance improvements. While DOD in the gaming industry is not specifically tied to applications of AI, its core principles are remarkably similar to those of DOA.

A growing body of literature focuses on surveying the application of AI. Table 1 shows how our work compares against these surveys. One category of current surveys presents AI applications that solve problems in specific domains. For example, Cai et al. [29] survey ML techniques applied to the healthcare domain. They review systems for disease diagnosis and treatment. Bohg et al. [19] survey data-driven methodologies for robot grasping. They focus on techniques based on object recognition and pose estimation for known and unknown objects. The review compares data-driven and analytics-driven approaches. Qin et al. [109] analyse ML methods for industrial fault detection and diagnosis. This survey focuses on fault detectability methods for industrial processes with different complexities. Wong et al. [143] present the challenges of applying **deep learning (DL)** techniques in radio frequency applications. This work reviews DL applications in the radio frequency domain from the perspective of data trust, security, and hardware/software issues for DL deployment in real-world wireless applications. Joshi et al. [64] survey the deployment of DL approaches at the edge. Their review presents architectures of DL models, enabling technologies, and adaptation techniques. Their work also describes different metrics

¹We refer to “ML-based systems in the real world” as software systems that deliver ML-powered capabilities, deployed and evaluated in production environments with real users, unpredictable physical processes, and unseen variable data. Such a definition contrasts with isolated academic settings with fixed datasets, simulated users, and environments, and tightly controlled experimental conditions.

Table 1. Related Work Compared Against This Survey

Related work	Domain	Focus on Architecture Style	Focus on Deployment	Focus on Data	Focus on Adoption
Cai et al. [29]	Healthcare	No	Yes	No	No
Bohg et al. [19]	Robotics	No	Yes	No	No
Qin et al. [109]	Industrial processes	No	Yes	No	No
Wong et al. [143]	Radio frequency	No	Yes	No	No
Joshi et al. [64]	Edge computing	No	Yes	No	No
Muccini and Vaidhyanathan [85]	ML software architecture	Yes	Yes	No	No
Washizaki et al. [140]	ML engineering	Yes	No	No	No
Chai et al. [32]	ML pipelines	Yes	No	No	No
Giray [53]	ML engineering	Yes	Yes	No	No
Heck [59]	Data engineering	No	Yes	Yes	Yes
This work	ML-based Systems	Yes	Yes	Yes	Yes

Our work is the only survey that measures the adoption of an architectural style (i.e., DOA) in the context of ML-based systems at deployment.

for designing and evaluating DL models at the edge. Our survey is domain-agnostic and focuses on articles that report ML-based systems at deployment. We extend specific domain surveys by analysing the design decisions and software architectural patterns behind the systems the authors report.

Another category of articles reviews software engineering patterns for ML applications. Since DOA is a general architectural style that builds on multiple design patterns; our survey is closely related to these works. Muccini and Vaidhyanathan [85] emphasise the importance of research on architectural patterns for ML-based systems and provide high-level research directions. Washizaki et al. [140] discuss the patterns for building ML-based systems in-depth by reviewing academic literature and engineering blog posts, and identify 33 patterns. DOA directly applies or encourages many of these patterns (e.g., “Kappa Architecture” and “Modularisation of ML Components”). Chai et al. survey challenges that arise at different stages of ML pipelines and acknowledge data management as one of the crucial challenges for ensuring the high quality of a pipeline [32]. Giray [53] performs a systematic analysis of software engineering practices for ML-based systems. Heck [59] conducts a mapping study on data engineering for AI systems, briefly summarising relevant technical solutions and architectural styles. At a high level, these surveys describe multiple architectural styles, patterns, and paradigms for designing and developing ML-based systems. They attempt to deepen the community’s understanding of modern software engineering for ML. We provide further insights by exclusively reviewing ML-based systems from a data-oriented perspective (i.e., data focus), which is the crucial and distinct aspect of modern and future data-driven systems. Moreover, none of the articles in the related work perform a detailed study of a single architectural style or pattern in the context of ML-based systems. In contrast, our work surveys the adoption level of DOA when deploying ML-based systems. We define the core principles of DOA, compare them against ML challenges at deployment, and provide advice on applying DOA in practice. We quantify the adoption of DOA principles among already existing ML-based systems by conducting a deep architectural analysis of these works. Our long-term goal is to unify multiple approaches for deploying ML-based systems under the DOA architectural style.

3 DOAs and ML Challenges

SOA is the prevalent architectural style for implementing modern software. It defines services as individual components that expose their functionalities through interfaces (APIs) [130]. Modularity enables the assignment of system components to different teams, allows flexible scaling, and facilitates maintainability. Independent departments in the organisation focus on specific services instead of the whole system because of the well-defined interfaces between the parts of the system [70, 86]. In practice, systems built following SOA are usually deployed on cloud platforms for high availability and robustness. Flexible and automatic management of resources (replicas, load balancers, etc.) enables systems to be fault-tolerant and guarantees the quality of service despite the amount of data being processed [67]. Maintainability, high availability, and scalability are essential requirements that ML-based systems must satisfy when deployed in the real world. Service abstractions and cloud deployments enable current system architectures to meet these requirements. However, real-world environments pose additional requirements on ML-based systems, challenging SOA practices. Data monitoring and transparency are challenging because services hide data behind their interfaces (“The Data Dichotomy” [130]). Service interactions based on interfaces prioritise control flow and make the data exchanged during the communication ephemeral [99]. Cloud platforms are expensive and complicate ownership of end users’ data. The physical distance between end users and cloud servers impacts the end-to-end latency of critical applications supported by AI (e.g., real-time health diagnosis) [28]. Cloud resources are usually expensive, failing to satisfy resource constraints when budgets are limited in real-world systems [75, 122]. Cloud deployments can imply a change in data ownership from end users to service providers. End users are unaware of where and how providers process their sensitive data and the associated security risks. Data ownership, security, accountability, and trust requirements are hard to satisfy in such deployments [4, 34, 42, 74].

DOA is an emerging architectural style that can help software engineers overcome these challenges when deploying ML-based systems. DOA proposes a set of principles for implementing data-oriented software: data as a first-class citizen, decentralisation as a priority, openness [65, 74, 120, 138]. Figure 1 maps ML deployment challenges and the DOA principles that address them. The left side of the figure shows the challenges of the ML workflow at deployment discussed by Paleyes et al. [105], while the right side shows the DOA principles we have extracted from the literature [65, 74, 120, 138]. This section introduces each DOA principle and discusses how these principles support the deployment of ML-based systems.

3.1 Data as a First-class Citizen

Data collection, preprocessing, analysis, and monitoring tasks are challenging for systems deployed in the real world. Such environments usually generate high volumes of variable data, which need to be processed in real time to create value. These requirements come from the properties of the data real-world systems must handle, which are also known as “the five Vs” of big data: volume, variability, velocity, veracity, and value [88]. Such data properties and requirements become more critical for the implementation of ML-based systems because they are inherently data-driven. The outputs and overall performance of ML-based systems depend on the quality of the data that flows through their components [46]. Practitioners must design data-driven systems focusing on the specific data to be processed. These systems must monitor their data to evaluate performance at runtime, identify or predict failures, and trigger maintenance or adaptation operations. Architectural styles focused on individual services encapsulate atomic capabilities and interact through their interfaces, falling into “the Data Dichotomy”. These architectures hide their data behind service interfaces, but data management requires exposing data [130]. The data dichotomy does not suit data management tasks

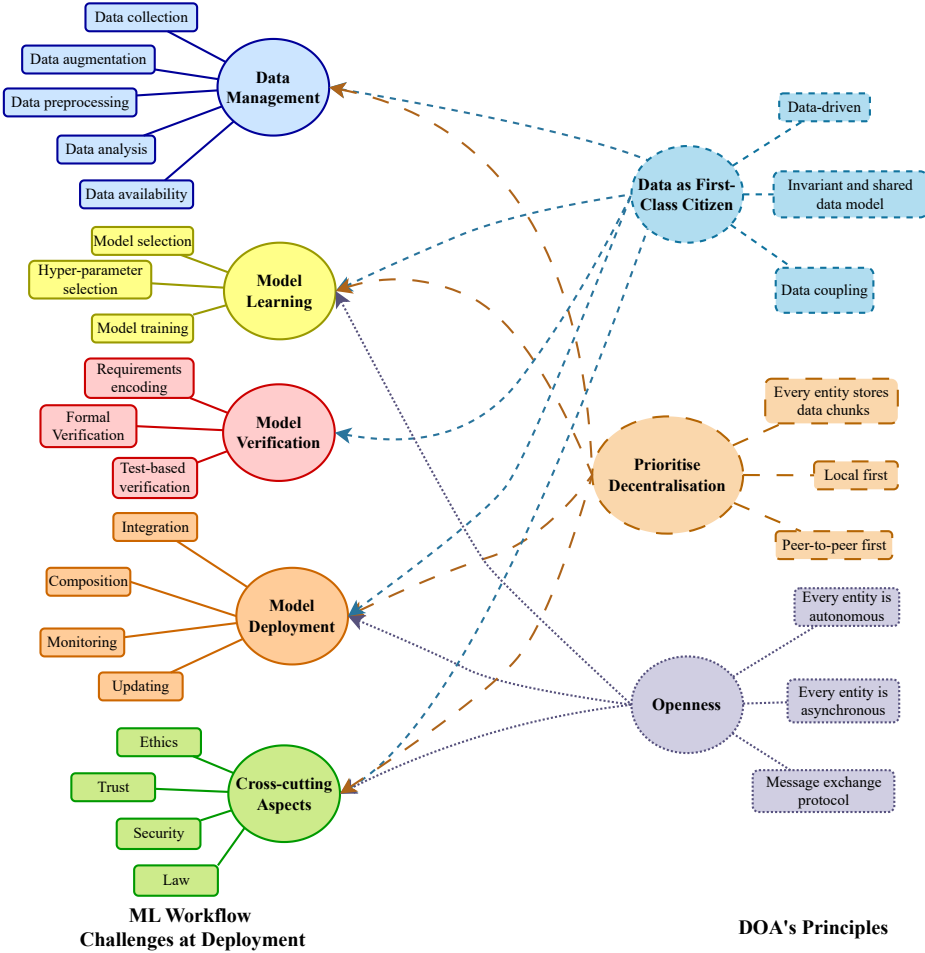


Fig. 1. Map between ML challenges at deployment [105] and DOA principles. The left side shows the ML challenges, and the right side shows the DOA principles. The links between them represent which principles support addressing the respective challenges.

because additional efforts are needed to access the data that flows through the system. Such data unavailability complicates systems' monitoring, transparency, and related objectives [34, 148]. In the ML context, it means that systems' data unavailability challenges the selection, parametrisation, training, testing, verification, integration, monitoring, composition, and updating of the learning model components as well as systems' data transparency requirements [105].

DOA proposes to treat the **data as a first-class citizen** of a software system, understanding data as the common denominator between disparate components [65]. The data in a DOA-based system is primary, and the operations on the data are secondary. This principle makes systems *data-driven* by design, which matches the nature of ML-based components. DOA-based systems rely on a *shared data model*, processed and nourished by multiple components [120]. The shared data model is a single structure equivalent to the one that data engineers build into the data

management stage of ML workflows. The key difference is that the system automatically creates this data model from the interaction of its components. System components do not expose any APIs and interact via data mediums, where input is listened to and output is offloaded [120]. Such interactions enable DOA-based systems to achieve *data coupling*, considered the loosest form of coupling [92, 93]. The shared data model stores the history of the system's state during its whole life cycle. Systems' data is fully available, describing the systems' current and past states. Data availability facilitates data management, learning model management, systems monitoring, failure detection, and adaptation tasks. Components' behaviour is programmatically observable, traceable, and auditable. Such transparency helps the design and implementation of ML-based systems to resolve questions regarding accountability, trust, and traceability.

3.2 Prioritise Decentralisation

Big tech companies drove recent ML breakthroughs thanks to the increasingly powerful hardware and the efforts in building massive datasets [41, 77]. These enablers are not always present in systems deployed in the real world. Smaller organisations are using ML to solve challenging problems with limited budgets and resources.² Most organisations worldwide cannot afford expensive computing resources or the effort of building and maintaining massive learning models. Such a reality threatens the practical and democratic adoption of ML models, as different stages of their workflow are computationally expensive. For example, the training stage is an iterative process that solves an optimisation problem to find learning model parameters. Complex and large models make this process computationally expensive when, for example, they learn from non-convex high-dimensional datasets [54, 66, 96]. Hyperparameters improve the efficiency of the training process and the accuracy of the learning models [55]. However, selecting these hyperparameters is also a resource-demanding optimisation problem [17, 98]. In addition, expensive and physically centralised deployments can fail to meet low-latency requirements from critical applications. The physical distance between end users and data centres impacts systems' end-to-end response time [28]. Data ownership, security, and trust requirements are also affected in centralised systems. The ownership of the data changes from end users to service providers, which can entail privacy and security issues as end users are unaware of where and how their data is stored and processed [4]. Recent research shows that **Large Language Models (LLMs)** create risks of leaking sensitive information from the training data. Adversarial attacks or prompt engineering strategies can extract personal information from LLMs [42].

The simplest way to deliver a shared data model (Principle 3.1) would seem to be to centralise it, but in practice requirements for scalability, resources, low latency, and security mean that in DOA we **prioritise decentralisation** [74, 138]. Such decentralisation should be logical and physical. Logical decentralisation enables organisations to scale when developing ML-based systems, as different development teams focus on smaller system components. Logical decentralisation is a way to achieve separation of concerns, avoid central bottlenecks, and increase flexibility in system development. Physical decentralisation enables the deployment of ML models in constrained environments without expensive resources, and improves privacy and ownership of data [74]. Practitioners should deploy components of ML-based systems as decentralised *entities that store data chunks* of the shared information model described above. These entities first perform their operations with their local resources (i.e., *local first*, [74]). If local resources (i.e., data, computing time, or storage) are insufficient, entities connect temporarily with other participants to share resources. Entities first scan their local environment for potential resources they need. They prioritise interactions with nodes in close vicinity to share or ask for data and computing resources

²An example of such an organisation is **Data Science Africa (DSA)**: <https://www.datascienceafrica.org/>

(i.e., *peer-to-peer first*). Centralised servers serve as a fallback mechanism [138]. This principle facilitates data management, enabling the system's data replication by design because different entities can store the same data chunk. Such replication provides data availability because if one entity fails, its information is not lost. Similarly, replication provides scalability as different entities can respond to concurrent data requests [138]. Prioritised decentralisation also alleviates the high demand for resources of ML-based systems and reduces the inference latency. ML-based systems can perform their data-related tasks (e.g., ML model training) on smaller datasets partitioned by design, and deploy trained models closer to end users to provide faster responses [24]. In addition, decentralisation creates an ecosystem where we can use resources from different devices on demand. This DOA principle advocates for a more sustainable and democratic approach that prioritises the computational power available in everyday devices over expensive cloud resources. It is important to note that logical or physical decentralisation is not always necessary. However, data replication, partitioned datasets, and flexible resource management are DOA-enabled properties that can still benefit even from partial decentralisation of ML-based systems.

3.3 Openness

Developing ML components often involves automating the training and inference stages of data processing. The amounts of data ML-based systems manage, the complex processes they perform, and the fact that their users are usually experts in domains different from ML (e.g., healthcare, physics, and engineering) [139] demand a high level of automation. AutoML is a research area that aims to automate the ML training life cycle. AutoML approaches include data augmentation, model selection, and hyperparameter optimisation [43, 136]. Nevertheless, implementing ML-based systems in real-world environments sets automation requirements for adopting ML models that AutoML does not cover. Real-world environments are dynamic, and deployed systems must respond autonomously to changing goals, fluctuating data distributions, unexpected failures, security threats, and complex uncertainties. Autonomous responses are required because practitioners do not have complete control over such deployments [97], and overall system complexity can exceed the limit of individual human comprehension [100], making human intervention unfeasible [25]. Current ML-based systems rely on the interaction of heterogeneous components. Static architectures usually predefine the components and devices that compose a system, and third-party entities (e.g., central controllers) integrate, orchestrate, and maintain these components to satisfy end-to-end system quality requirements [26]. The scale and dynamic nature of real-world deployments demand autonomous solutions and challenge these static and centralised architectures.

Autonomous and independent software components are the building blocks of systems that can adapt and respond to uncertain environments without human intervention [142]. DOA proposes **openness** as a principle that enables the design and implementation of independent system components in open environments where they interact asynchronously [65]. DOA describes software components as *autonomous* and *asynchronous* entities that communicate with each other using a common *message exchange protocol*. Systems can leverage such environments to integrate ML models because their decentralised components can autonomously perform the integration, composition, monitoring, and adaptation tasks. Asynchronous entities produce their outputs and can subscribe to inputs at any time [120]. These steps are transparent, favouring data trust and traceability. Similarly, the system's components are autonomous in deciding which data to store, which to make public, and which to hide for security and privacy [138]. The message exchange protocol between asynchronous components replaces interface dependencies in service-oriented or microservices architectures with asynchronous messages between data producers and consumers. Asynchronous communication protocols enable data coupling, considered the loosest form of coupling, enabling scalability and low latency [92, 93].

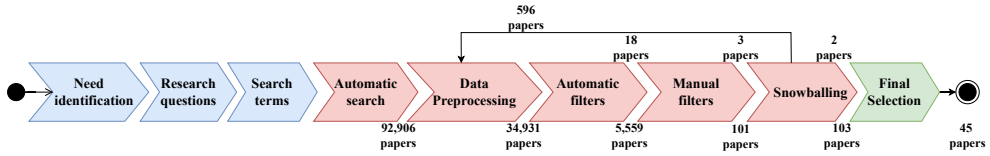


Fig. 2. The survey process includes the steps from the review motivation to full-text reading and the final selection of articles for analysis. This process relies on the methodology proposed by Kitchenham et al. [72, 73].

4 Survey Methodology

This survey assessed why, how, and to what extent practitioners adopted the DOA principles when deploying ML, as well as the motivation and methods behind such adoption. We survey research works that used ML to solve problems in different domains and were deployed in real-world settings. We are interested in works that report the software architectures behind these systems and, when described, the authors’ design decisions toward such architectures.

Selecting relevant works for our survey was not straightforward because many articles applying ML in different domains had been published in recent years, making manual selection unfeasible. For this reason, we developed a semi-automatic framework based on a well-known methodology for **systematic literature reviews (SLRs)** in software engineering [72, 73]. The framework is available publicly on GitHub³ to enable reproducibility of this work, and the reusability of this framework in other surveys. The framework queries the search APIs from different digital libraries to automatically retrieve articles’ metadata (e.g., title, abstract, and citations). It then applies syntactic and semantic filters over the retrieved records to reduce the search space. Then, we manually filter the list of works to select the articles to survey. Figure 2 depicts the stages of the survey process. Its two principal stages are described in the next section.

4.1 Planning Stage

The first stage consists of the review plan definition as follows:

- Need identification*: Section 3 introduced the DOA principles and how these support practitioners when deploying ML in the real world. Despite these potential benefits and several surveys on ML (see Section 2), it remains unclear to what extent, why, and how practitioners have adopted these principles in deploying ML-based systems. Practitioners usually follow DOA principles implicitly, creating a gap that obscures the software development practices, system design patterns, and architectural styles practitioners apply when building ML-based systems. Closing this gap will contribute to proposing and developing novel engineering approaches and system architectural styles for ML-based systems, which are fundamentally different from traditional software systems [23, 105]. We start closing such a gap by surveying deployed ML-based systems from a DOA perspective to identify the design decisions, implementation practices, and deployment strategies practitioners followed.
- Research questions*: The main research question of this survey is: *To what extent, and how have researchers adopted the DOA principles in the implementation and deployment of ML-based systems, and what were the motivations behind such adoption?* We split this question into three complementary research questions to determine the level and way of adoption of each DOA principle: **data as a first-class citizen (RQ1)**, **prioritise decentralisation (RQ2)**, and **openness (RQ3)**. The answers to these questions enable us to identify the research gaps and

³Semi-automatic Literature Survey: <https://github.com/cabrerac/semi-automatic-literature-survey/tree/doa-survey>

Table 2. Search Query Format Used to Retrieve Articles

Search Query	<search_term_1> AND <search_term_2> AND <search_term_3>
<search_term_1>	“autonomous vehicle” OR “health” OR “industry” OR “smart cities” OR “multimedia” OR “science” OR “robotics” OR “oceanology” OR “finance” OR “space” OR “e-commerce”
<search_term_2>	“machine learning”
<search_term_3>	“real world” AND “deploy”

The query is composed of three search terms in conjunction. The values in the second column replace each search term.

Table 3. Synonyms to Extend Queries

Word	Synonyms
“health”	“healthcare”, “health care”, “medicine”, “medical”, “diagnosis”
“industry”	“industry 4”, “manufacture”, “manufacturing”, “factory”, “manufactory”, “industrial”
“smart cities”	“sustainable city”, “smart city”, “digital city”, “urban”, “city”, “cities”, “mobility”, “transport”, “transportation system”
“multimedia”	“virtual reality”, “augmented reality”, “3D”, “digital twin”, “video games”, “video”, “image recognition”, “audio”, “speech recognition”, “speech”
“science”	“physics”, “psychology”, “chemistry”, “biology”, “geology”, “social”, “maths”, “materials”, “astronomy”, “climatology”, “oceanology”, “space”
“autonomous vehicle”	“self-driving vehicle”, “self-driving car”, “autonomous car”, “driverless car”, “driverless vehicle”, “unmanned car”, “unmanned vehicle”, “unmanned aerial vehicle”
“networking”	“computer network”, “intranet”, “internet”, “world wide web”
“e-commerce”	“marketplace”, “electronic commerce”, “shopping”, “buying”
“robotics”	“robot”
“finance”	“banking”
“machine learning”	“ML”, “deep learning”, “neural network”, “reinforcement learning”, “supervised learning”, “unsupervised learning”, “artificial intelligence”, “AI”
“deploy”	“deployment”, “deployed”, “implemented”, “implementation”, “software”
“real world”	“reality”, “real”, “physical world”

Search terms in the Word column are expanded using their respective synonyms.

inform the development of the next generation of DOAs. Our long-term goal is to establish DOA as a mature architectural style for designing and deploying ML-based systems.

- (c) *Search terms:* We searched for articles presenting ML-based systems deployed in real-world environments to answer the research questions. Table 2 shows the query format and the search terms we use to retrieve such articles. The query has three search terms in conjunction. The first term refers to popular domains that have applied ML. The second term filters articles that apply ML in these domains, and the third term filters articles that deploy their solution in the real world. Search engines in scientific databases use different matching algorithms. Some engines search for exact words in the articles’ attributes, which can be too restrictive. We expand these queries by including synonyms for the search terms (Table 3). Synonyms extend queries using inclusive disjunction.
- (d) *Source selection:* We searched for articles in the most popular scientific repositories using their APIs. They are IEEEExplore,⁴ Springer Nature,⁵ Scopus,⁶ Semantic Scholar,⁷ CORE,⁸

⁴IEEEExplore API: <https://developer.ieee.org/>

⁵Springer Nature API: <https://dev.springernature.com/>

⁶Scopus API: <https://www.elsevier.com/solutions/sciencedirect/librarian-resource-center/api>

⁷Semantic Scholar: <https://www.semanticscholar.org/product/api>

⁸CORE API: <https://core.ac.uk/services/api>

Table 4. We Use the Following Categories and Keywords for the Lbl2Vec Algorithm [118]

Category	Keywords
“system”	“architecture”, “framework”, “platform”, “tool”, “prototype”
“software”	“develop”, “engineering”, “methodology”, “architecture”, “design”, “implementation”, “open”, “source”, “application”
“deploy”	“production”, “real”, “world”, “embedded”, “physical”, “cloud”, “edge”, “infrastructure”
“simulation”	“synthetic”, “simulate”

and ArXiv.⁹ Some repositories, such as the ACM digital library, could not be used as they do not provide an API to query. Nevertheless, we are confident in the coverage of our search because of the significant overlap with other sources.

4.2 Conducting Stage

Table 5 shows the number of articles our process filters at each step of the conducting stage per repository. The rest of this section details these steps and how we filter these works.

- Automatic search:* We implemented clients that consume the repositories APIs as part of our framework. Each client creates the query in the respective API format, submits the request, and stores the search results in separate CSV files. The search results are the metadata of the retrieved articles (e.g., title, abstract, and publication date).
- Preprocessing of Retrieved Data:* Each API provides articles’ metadata in a different format. There were duplicate articles between repositories, and some records were incomplete. The preprocessing step prepares the data for the following steps in our semi-automated framework. It merges the articles’ metadata into a single file, cleans the data, and removes repeated and incomplete records. The process selected a total of 34,931 works after this step.
- Automatic Filtering:* All the data of the retrieved articles are stored in a single file after the preprocessing step. However, the number of articles is still too large for manual processing. We reduce the search space by applying two filters. A syntactic filter selects the articles that mentioned real-world deployments in the abstract according to our definition of “ML systems in the real world”. In particular, this filter searches for the “real world” and “deploy” words and their synonyms (Table 3) in the articles’ abstracts. We classify the selected articles into four categories after the syntactic filtering. The first category includes the articles that present architectures of deployed ML-based systems, the second category groups articles that present software engineering approaches to build ML-based systems in practice, and the third category represents articles that show physical implementations (e.g., edge architectures) of ML-based systems with a special focus on the infrastructure. The final category includes articles that evaluate ML algorithms and systems based on synthetic data and simulated environments. We use an unsupervised-learning algorithm, Lbl2Vec, to semantically classify the selected articles in these four groups following the work proposed by Schopf et al. [118]. Lbl2Vec requires a set of texts to classify as inputs (i.e., selected articles’ abstracts) and a set of predefined categories (Table 4). The algorithm assigns articles to the most relevant category. We use this semantic classification to select the articles that belonged to the first three categories (i.e., system, software, and deploy). These filters produced a total of 5,559 articles.

⁹ArXiv API: <https://arxiv.org/help/api/>

mediums (i.e., decentralisation and openness). For each of these columns, we define three possible outcomes:

- **“YES”** meaning the reviewed article fully follows the evaluated principle. For example, a system fully follows the data as a first-class citizen principle if the system is data-driven, its components share a data model, and they communicate through this data model (i.e., data coupling).
- **“PARTIAL”** meaning the reviewed article partially follows the evaluated principle. For example, a system partially prioritises decentralisation if it stores and processes data chunks locally but does not have peer-to-peer communication with components in the same layer.
- **“NO”** meaning the reviewed article does not follow the evaluated principle in any way. For example, a system does not follow the openness principle when it does not implement autonomous and asynchronous entities exchanging messages between them.

Each researcher justifies the outcome for each column and article in the data collection spreadsheet. In addition, the spreadsheet also has a column “Comments” to annotate extra information researchers find relevant for our study. Selection and data extraction conflicts were addressed in group discussions. Section 5 presents a survey of the 45 articles using a descriptive and quantitative synthesis of the results we extracted in the spreadsheet.

(g) *Exclusion criteria:* We exclude the following works during the different stages of our semi-automatic process:

- Articles that introduce ML-based systems without deployment and evaluation in the real world or a production environment.
- Articles that present ML-based systems but do not report their software architecture.
- Articles that report experiments on synthetic data or simulated environments where ML-based systems do not interact with actual data, physical entities, or people.
- Articles presenting isolated ML algorithms not part of larger systems.
- Articles with missing metadata that our framework cannot analyse (e.g., an article without an abstract).
- Duplicated articles.
- Articles not written in English.
- Survey and review articles.
- Thesis and report documents.

We use the exclusion criteria to distinguish ML-based systems deployed in the real world from ML systems or algorithms developed and tested in controlled and isolated environments.

5 ML-based Systems Survey from the DOA Principles Perspective

This section presents a survey of the selected ML-based systems. Our main goal is to understand why, how, and to what extent practitioners have adopted the DOA principles (Section 3). Answers to these questions allow us to identify the systems’ requirements that DOA principles satisfy and the practical approaches for implementing DOA, as well as to define the open challenges for building DOA-based systems (Section 6). Table 6 shows the systems that adopt the DOA principles by contrasting them with the list of reviewed articles. The ML-based systems reported in these articles fully, partially, or do not adopt the DOA principles. The level of adoption depends on the requirements these systems need to satisfy, the nature of the data they handle, and their deployment environments. This section details the adoption of each principle and sub-principle. Table 6 shows that nine out of the 45 articles follow all or most DOA principles, reflecting an overall low adoption of DOA when implementing current ML-based systems. The lack of maturity and tooling around the DOA, in contrast with popular SOA cloud-based deployments, is the reason behind this trend.

Table 6. All Reviewed Articles in Our Survey

Research work	Data as a First Class Citizen			Prioritise Decentralisation			Openness		
	Data driven	Shared data model	Data coupling	Local data chunks	Local first	Peer-to-peer first	Autonomous entities	Asynchronous entities	Message exchange protocol
Lebofsky et al. [76]	✓	✓	✓	✓	✓	✓	✓	✓	✓
Herrero et al. [61]	✓	✓	✓	✓	✓	✓	✓	✓	✓
Zhang et al. [146]	✓	✓	✓	✓	✓	✓	✓	✓	✓
Dai et al. [37]	✓	✓	✓	✓	✓	✓	✓	✓	✓
Junchen et al. [62]	✓	✓	✓	–	✓	✓	✓	✓	✓
Karageorgou et al. [68]	✓	–	✓	✓	✓	✓	✓	✓	✓
Shan et al. [124]	✓	✓	✓	✓	✓	✓	✓	–	–
Zhang et al. [147]	✓	✓	–	✓	✓	✓	–	–	–
Sultana et al. [131]	✓	✓	✓	–	–	–	✓	✓	✓
Santana et al. [116]	✓	✓	–	–	–	–	–	–	–
Alonso et al. [8]	✓	–	–	–	–	–	–	–	–
Sarabia-Jácome et al. [117]	✓	–	–	–	–	–	–	–	–
Xu et al. [144]	✓	✓	–	✓	–	✓	✓	–	–
Alves et al. [9]	✓	✓	✓	–	–	–	–	–	–
Conroy et al. [35]	✓	✓	✓	–	–	–	–	–	–
Bellocchio et al. [15]	✓	✓	–	–	–	–	–	–	–
Shih et al. [127]	✓	✓	–	–	–	–	–	–	–
Gallagher et al. [49]	✓	✓	–	–	–	–	–	–	–
Salhaoui et al. [114]	✓	–	–	–	–	–	–	–	–
Shi et al. [126]	✓	–	–	–	–	–	–	–	–
Nguyen et al. [89]	✓	✓	✓	–	–	–	–	✓	✓
Schumann et al. [121]	✓	✓	✓	–	–	–	–	✓	✓
Schubert et al. [119]	✓	✓	✓	–	–	–	–	✓	✓
Agarwal et al. [2]	✓	✓	✓	–	–	–	–	–	–
Habibi et al. [57]	✓	✓	–	–	–	–	–	–	–
Müller et al. [87]	✓	✓	–	–	–	–	–	–	–
Lu et al. [79]	✓	✓	–	–	–	–	–	–	–
Calancea et al. [30]	✓	–	–	–	–	–	–	–	–
Quintero et al. [111]	✓	–	–	–	–	–	–	–	–
Franklin et al. [48]	✓	–	–	–	–	–	✓	✓	✓
Brumbaugh et al. [21]	✓	–	–	–	–	–	–	–	–
Hegemier et al. [60]	✓	–	–	–	–	–	–	–	–
Gorkin et al. [56]	✓	–	–	–	–	–	–	–	–
Barachi et al. [13]	✓	–	–	–	–	–	–	–	–
Niu et al. [91]	✓	–	–	–	–	–	–	–	–
Qiu et al. [110]	✓	–	–	–	–	–	–	–	–
Cabanes et al. [22]	✓	✓	✓	–	–	–	–	–	–
Gao et al. [50]	✓	✓	–	–	–	–	–	–	–
Amrollahi et al. [10]	✓	–	–	–	–	–	–	–	–
Kemsaram et al. [69]	✓	✓	–	–	–	–	–	–	–
Bayerl et al. [14]	✓	–	–	–	–	–	–	–	–
Johnny et al. [63]	✓	–	–	–	–	–	–	–	–
Falcão et al. [44]	✓	✓	–	–	–	–	–	–	–
Hawes et al. [58]	✓	✓	–	–	–	–	–	–	–
Ali et al. [7]	✓	–	–	–	–	–	–	–	–

This table shows the extent to which and in which articles practitioners adopt (fully or partially) each of the DOA sub-principles discussed in Section 3.

✓ = Adopted, – = Partially adopted,  = Not adopted

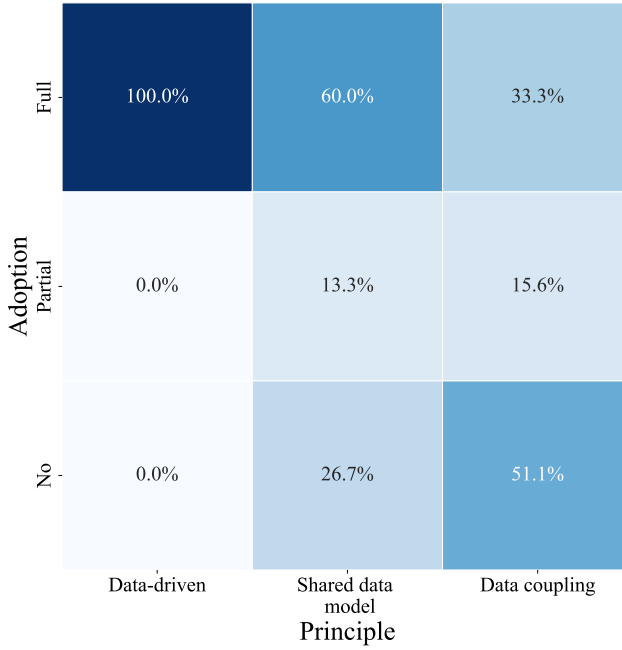


Fig. 3. Data as a first-class citizen principle adoption. All reviewed systems are data-driven, over 60% at least partially adopt a shared data model, and just under half utilise data coupling.

5.1 Data as a First-Class Citizen

Figure 3 shows the extent to which the reviewed articles adopt the **data as a first-class citizen** principle. We found that all reviewed articles report *data-driven* systems, which is an expected result because ML-based systems are *data-driven*. All the surveyed systems contain one or multiple learning models because they need to satisfy data needs, such as identifying anomalies, optimising operations, automating processes, and so on. These systems' performance depends on the data flowing through them, and they fulfil their requirements using diverse data-driven methods. For example, Schumann et al. [121] use a Bayesian network for space vehicles monitoring (e.g., rovers), Sarabia-Jacome et al. [117] build a DL model for fall detection in **Ambient Assisted Living (AAL)** environments, Junchen et al. [62] propose Pytheas as a data-driven approach that optimises the **Quality of Experience (QoE)** of applications based on network metrics, Agarwal et al. [2] present an RL-based decision-making system that Microsoft products use for different optimisation tasks, such as content recommendation systems in MSN and virtual machine management in Azure Compute.

Our findings show that 60% of the surveyed articles fully adopt the sub-principle of handling their information using *shared data models*. These systems implement *shared data models* using data streams or databases. Practitioners select data streams because these structures enable systems to manage and process information from continuous data sources. That is the case of environments that require systems to process multimedia streams (e.g., video) [37, 44, 50, 57, 62], sensor data [22, 35, 61, 69, 79, 89, 121, 127, 147], social media data [87, 124, 144, 146], and network metrics [116, 131]. We observed that streams are common in systems built with a dataflow architecture [36, 99]. A dataflow architecture transforms data inputs using different components while they flow through

the system. Herrero et al. [61] present a good example of a system based on a dataflow architecture. They propose a platform for Industry 4.0 based on the RAI4.0 reference architecture [81] that models software and hardware components as stream producers and consumers. This platform handles data-intensive, continuous, and dynamic data sources from a water pump and its environment. Streams enable the system to manage and respond to real-time changes while providing predictive maintenance. Similarly, Sultana et al. [131] present a software framework to detect **Distributed Denial of Service (DDoS)** attacks in real-time network traffic. This software relies on a **Support Vector Machine (SVM)** model deployed using the Acumos and ONAP platforms. The framework decouples components that exchange data between them via Kafka streams. System designers prefer databases as *shared data models* when the system's functionalities require making decisions based on historical data records [2, 9, 15, 49, 58, 76, 119]. The reason behind this design decision is that a shared data model implemented as a database facilitates the storage and management of historical records and provides an efficient communication medium between systems' components compared to synchronous API calls or **Remote Procedure Calls (RPCs)**. For example, Alves et al. [9] propose a system for industrial predictive maintenance based on historical data collected from IoT devices. These devices produce and write monitoring data in a database. The system's components read such data for the respective maintenance decision-making. Schubert et al. [119] introduce a DL-based system used by the Texas Spacecraft Laboratory. The system's components store and retrieve data from AWS S3 buckets to generate large datasets of synthetic images that support on-orbit spacecraft operations. Hawes et al. [58] present a robotic platform that stores the **Robot Operating System (ROS)** messages in a document-oriented database (i.e., MongoDB). These messages describe the status of the robot and its interactions with the environment. A monitoring component reads these historic messages to predict future states. This robotic platform is the only reviewed work that uses the *shared data model* and *data coupling* principle for self-monitoring purposes.

Over 13% of the articles partially adopt a *shared data model* between their systems' components. That is the case for systems combining heterogeneous storage technologies in shared data models [10, 21, 68]. These systems must store, monitor, and trace the data status in different stages of its processing. Practitioners design and implement systems combining storage technologies and models because the data structure and format change after each processing stage. For example, the AIDEx platform [10] queries an **electronic medical records (EMRs)** database and passes such data to a set of microservices that predict the risk of patients' infection sepsis based on their data models. AIDEx creates patients' health data streams based on the prediction results stored in a MongoDB instance. The system uses this data for visualisation purposes. Karageorgou et al. [68] propose a system for multilingual sentiment analysis of X (formerly known as Twitter) streams. This platform uses RabbitMQ queues to collect tweets in different languages. The system then models such data as Kafka streams for sentiment analysis. Other systems that partially adopt a *shared data model* are the ones that must collect data from spatially distributed sources [8, 14, 117]. Systems distribute such data to achieve low latency, satisfy resource constraints, and privacy requirements. Distributed nodes use the same models to store local data, but do not share this data between themselves. Section 5.2 reports more details on distributed and decentralised deployments.

Just over a quarter of the reviewed articles report systems that do not have a shared data model. Practitioners design systems without a shared data model because these systems are relatively small and not data-intensive. These are either implemented as monoliths in the cloud [56, 91] or as edge architectures that use edge servers as gateways for data transmission and inference. Such edge solutions deploy trained learning models on embedded devices, like in the monitoring and prediction systems proposed by Quintero et al. [111], Salhaoui et al. [114], Hegemier et al. [60],

and Johnny et al. [63]. Monolithic deployments follow a client-server pattern to collect sensor data and respond to requests. For example, Niu et al. [91] deploy a recognition system of indoor daily activities for one-person household apartments. Some systems do not have shared data models but are complex and process big volumes of data. These rely on cloud data centres and follow well-known software architectures (e.g., microservices) [7, 13, 30, 48, 110, 126]. Functionalities and data of systems' subcomponents are encapsulated and hidden behind interfaces (e.g., service APIs). Designers prefer such implementations when they do not prioritise transparency and traceability requirements and when they rely on third-party cloud components for monitoring tasks (e.g., middleware solutions [26]).

Systems' components in a third of the reviewed articles follow the *data coupling* sub-principle. These systems' components interact by reading from and writing to a data medium. Engineers adopt this principle because systems must handle real-time data that needs to be processed continuously. We found that components of real-time systems [21, 22, 35, 37, 61, 62, 68, 89, 121, 131] act as subscribers and publishers of data to streams that represent the state of the data at different stages in a workflow. Stream-based systems make use of different technologies such as Apache Kafka [61, 62, 131] or Spark Streaming [21, 62], sometimes adopting the underlying stream-based programming model for the entire system [37]. Message queues (e.g., RabbitMQ) are also used in systems to collect heterogeneous data from different sources or enable interaction between components in parallel [68, 89]. Systems also adopt the *data coupling* sub-principle to handle large amounts of data that are not possible to send through APIs or RPCs [76, 119] or when they analyse historical data [2, 9, 124, 146]. Practitioners design and implement data-coupled components using databases because systems and their components must process large pieces of data in batches. The system proposed by Zhang et al. [146] illustrates these types of *data coupling* by combining streams and databases. Distributed Apache Kafka streams store social media data from Weibo and Chinese forums, which are processed using Apache Storm to enable real-time sentiment analysis. The system also offers batch processing, where social media data is stored using the **Hadoop Distributed File System (HDFS)** and a HBase database. The Apache Spark **machine learning library (MLlib)** performs distributed data analysis on top of this data. Systems that partially adopt *data coupling* (i.e., 15.5%) [10, 21, 49, 50, 57, 87, 127] are the ones where some components communicate through data mediums and others use traditional service calls. The reason behind such a hybrid approach is satisfying interoperability requirements while managing large and real-time data. These systems use APIs or RPCs to communicate with distributed components or external entities, while the centralised system's components interact through data mediums. For example, systems can use API calls to collect data from heterogeneous data sources [10, 87] or to interact with end users [49, 50]. Over half of the surveyed articles do not adopt *data coupling*. These systems mostly use REST [13, 56, 63, 69, 79, 91, 111, 116] or RPC [8, 14, 15, 30, 44, 48, 58, 60, 110, 114, 117, 126, 144, 147] for communication. These systems' components hide the data behind their interfaces, which causes the data dichotomy issue impacting monitoring tasks [130].

We found that not all systems where components *share a data model* necessarily follow the *data coupling* sub-principle. Different nodes in a distributed system can have the same data model even when they do not interact. Designers make this decision when a system must address low latency and data privacy requirements, but does not need to aggregate data from geographically distributed sources for training ML models. Some of these distributed systems create independent deployments that are in charge of the functionality of the whole system in a given geographical region or coverage area [15, 58, 69, 79]. Other distributed systems rely on edge devices that play the role of intermediate nodes that collect, preprocess, and transmit data to centralised servers as well as perform inference tasks when designers deploy cloud-trained learning models on the edge [8, 14, 44, 116, 117, 124, 144, 147].

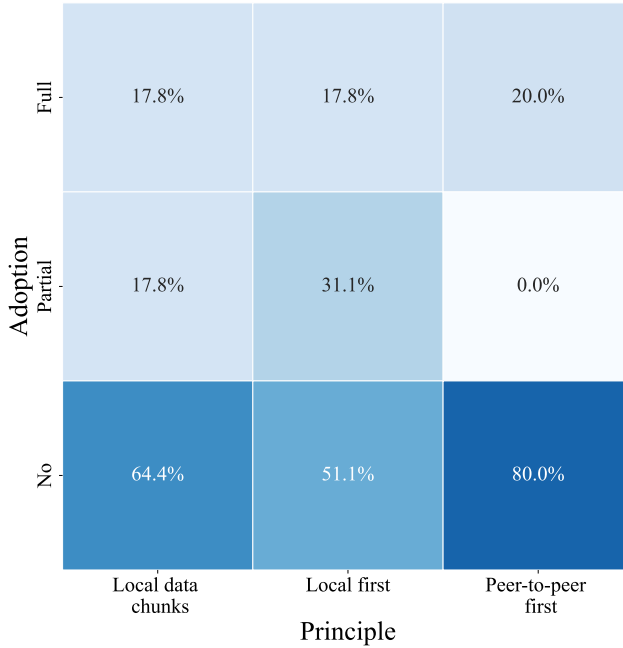


Fig. 4. Adoption of the prioritising decentralisation principle. Most systems rely on centralised storage and processing to the detriment of the decentralisation sub-principles.

5.2 Prioritise Decentralisation

Figure 4 presents the extent to which the articles **prioritise decentralisation** when deploying ML-based systems. Over 17% of the articles report systems where decentralised entities store *local data chunks* with minimal presence of centralised entities. Decentralised approaches [37, 61, 68, 76, 124, 144, 146, 147] rely on local data chunks, which provide partitioning and replication by design. Practitioners implement systems based on local data chunks to exploit such properties, increasing resources utilisation and fault tolerance. That is the case of the Breakthrough Listen programme presented by Lebofsky [76]. This programme collects data from radio and optical telescopes to **search for extraterrestrial intelligence (SETI)**. The search leverages different strategies, including ML models for modulation scheme classification and outlier detection. The article reports the software and hardware architecture behind the data collection, reduction, archival, and public dissemination pipeline. This architecture consists of storage clusters and compute nodes that handle raw data volumes averaging 1PB daily. Decentralised storage nodes use **Hierarchical Data Format version 5 (HDF5)** to organise the large datasets in groups that facilitate data management along the pipeline. Another example of decentralisation is Poligraph [124], a system that detects fake news by combining ML models and human knowledge. Users send fake news detection requests to decentralised servers that run different ML models and collect news reviews from experts. These servers run the **Byzantine Fault Tolerant (BFT)** protocol to replicate users' requests and news to mitigate experts' unavailability (i.e., fault tolerance). Systems designers also use decentralisation to implement systems for processing intensive and sparse data sources [61, 68, 144, 146]. For example, Zhang et al. [146] propose a system for sentiment analysis of tweets from four Chinese cities (i.e., Beijing, Shanghai, Guangzhou, and Chengdu). The decentralised storage

and management of data leverages HBase and Kafka. We found that decentralised approaches must rely on distributed computing protocols and technologies to mitigate the challenges that arise from the absence of a central control entity. In addition to the technologies mentioned before, Karageorgou et al. [68] use Spark for scalable sentiment analysis on multilingual data from social media, Xu et al. [144] use **Distributed Hash Tables (DHTs)** to facilitate the retrieval of social data for spam detection, and Herrero et al. [61] integrate Zookeeper, Kafka, and Apache Cassandra in a decentralised platform for a predictive maintenance industrial service.

Some systems partially decentralise the storage of data (17.8% of the articles). These approaches exploit decentralisation properties (e.g., closeness to data owners) while having centralised control of the system [106, 132]. Partial decentralisation creates federated networks where collected data from sensor devices (IoT sensors, smartphones, wearables, etc.) is stored in local databases [8, 15, 60, 126] or encoded in local learning models trained using more powerful computing infrastructure [60, 62, 114, 116, 117]. In both cases, edge servers preprocess and filter the collected data before transmitting it to back-end servers. Such central entities have a complete view of the system's state and perform more complex tasks (model training, job scheduling, resource allocation, etc.). System developers exploit these federated architectures to satisfy system requirements such as data ownership, privacy, and security, as devices and edge nodes can decide when and how to transmit the information to back-end servers. Santana et al. [116] propose a crowd management system based on Wi-Fi frames originating from people's smartphones in the context of the SmartSantander project [115]. The system collects Wi-Fi frames as streams preprocessed in edge nodes. This preprocessing includes data anonymisation and dimensionality reduction to protect people's identities and extract key features from the streams. A centralised processing tier stores and uses the preprocessed and anonymised data to train ML models (e.g., logistic regression, naïve Bayes, and random forest classifiers) for crowdsensing estimation. Practitioners also exploit local data storage to facilitate the deployment of ML-based solutions in resource-constrained environments and optimise cost. For example, Alonso et al. [8] implement a real-time monitoring edge system for smart farming, where farmers want to reduce storage and compute costs while monitoring the state of dairy cattle and feed grain. Edge servers in the system collect, buffer, and filter data from IoT devices to eliminate possible noise and discard duplicated frames. Such filtering also reduces transmission and storage costs.

Our findings show that most reviewed articles report systems that rely on centralised storage (i.e., 64.4%). Some of these systems are deployed on single cloud servers [2, 7, 10, 13, 21, 30, 35, 48–50, 56, 57, 79, 87, 91, 110, 119, 127, 131], where all data is stored. Designers rely on centralised cloud servers because these are flexible to handle big data requirements. For example, Conroy et al. [35] present a system that predicts COVID-19 infections to aid military workforce readiness. This system monitored personnel of the US Department of Defence during the pandemic. It had around 10,000 users in ten months, collected 201 million hours of data, and delivered 599,174 total user-days of predictive service. Similarly, Calancea et al. [30] use the Google Cloud service to store the data behind iAssistMe, a platform to assist people with eye disabilities. Both systems implement reactive scaling by increasing or reducing the number of servers based on the number of requests. Other systems are deployed on single resource-constrained devices like mobile phones [14, 111], robots [58, 69, 121], or small processing units (e.g., Raspberry Pi) [9, 22, 44, 63, 89]. Designers implement this type of deployment to fit data problems that require fast inference and actuation, but where data storage is not needed or infeasible due to hardware or bandwidth constraints. For example, Nguyen et al. [89] present a neuroprosthetic hand with embedded DL-based control. The authors deploy a **recurrent neural network (RNN)** decoder on an NVIDIA Jetson Nano, enabling the implementation of the neuroprosthetic hand as a self-contained unit with real-time control of individual finger movements. Data is not stored as it is processed online.

More than 17% of systems prioritise *local processing*. These systems deploy learning models and other components on distributed nodes that process users' requests, make decisions, and provide systems' functionalities. The main reason for decentralised processing is to meet scalability and low latency requirements in data-intensive applications [37, 61, 62, 68, 76, 124, 146, 147]. Dai et al. [37] introduce BigDL, a distributed DL framework for big data platforms and workflows. Intel developed the BigDL framework to ease the application of DL in real-world data pipelines for its industrial users: Mastercard, World Bank, Cray, Talroo, UCSF, JD, UnionPay, Telefonica, GigaSpaces, and more. These companies handle dynamic and messy data that require complex, iterative, and recurrent processing that is challenging to implement efficiently in centralised architectures. BigDL distributes training and inference on top of the scalable architecture of the Spark compute model. Spark partitions data across workers that form computing clusters. Workers apply map, filter, and reduce operations in parallel. The data partitioning and parallel processing enable the design and implementation of fault-tolerant and efficient systems based on learning models built from large datasets. A few cases offer systems' functionalities based on the cooperation of distributed nodes. These systems also follow the *peer-to-peer first* sub-principle and correspond to the 20% of the reviewed articles [37, 61, 62, 68, 76, 124, 144, 146, 147]. System designers adopt *local processing* and *peer-to-peer first* principles to satisfy scalability, latency, fault-tolerance, and resource management requirements of systems that handle data from intensive and distributed sources. That is the case with systems that analyse social media data, for example, sentiment analysis of tweets or spam detection [68, 144, 146], or the Breakthrough Listen program [76] in the SETI. Adopting the *peer-to-peer* sub-principle requires distributed computing technologies and protocols to handle the complexity of decentralised processing. Systems also use these technologies when they adopt the *local first* sub-principle. Examples of practical use of these technologies are Zhang et al. [146], Dai et al. [37], and Karageorgou et al. [68] using Spark, Junchen et al. [62] based on Kafka, Xu et al. [144] using DHTs, Lebofsky et al. [76] storing data on HDF5, Shan et al. [124] using the BFT protocol, and Herrero et al. [61] based on Zookeeper.

Over thirty-one per cent of the reviewed systems partially adopt the *local first* sub-principle. Nodes in the network are in charge of different functionalities in these systems. Centralised servers are in charge of computationally expensive processing such as model training and updating [14, 60, 63, 69, 111, 114, 117, 126, 144], global decision-making [8, 30, 116], and systems' orchestration [21, 131]. Centralised servers execute these tasks because they have enough computing power and a global view of the system. Central processing enables easier control of the components of the system, which can be challenging in fully decentralised systems, as autonomous entities must self-govern. Practitioners opt for such central control to make systems robust as they rely on centralised servers to offer scalable and highly available solutions. The works of Sultana et al. [131] and Brumbaugh et al. [21] are good examples of central control. They encapsulate systems' functionalities in logically distributed containers that Kubernetes centrally controls. Kubernetes is the central platform that manages container instantiation, resource allocation, scheduling, and execution. Systems that partially adopt the *local first* sub-principle use edge nodes or mobile devices to offer data collection and preprocessing functionalities. These nodes also offer inference capabilities when they host trained learning models. System developers use edge-computing architectures to enable local inference and fit low latency and data ownership requirements as learning models are closer to end users [28, 132]. For example, Sarabia et al. [117] deploy an edge-based fall detection system for AAL environments. AAL systems process highly sensitive patient data and require very low processing time. The authors propose a 3-layer fog-cloud architecture composed of medical devices, fog nodes, and a cloud server. DL models are deployed in fog nodes to detect patients' falls. The authors prove that the proposed platform performs better than a cloud baseline (i.e., better efficiency and response time). One or more fog nodes attend to each patient, creating a local area network to

preprocess (e.g., filtering) the patient's data. Dedicated fog nodes enable data privacy and security by design. Kemsaram et al. [69] present the architecture design and development of an onboard stereo vision system for cooperative automated vehicles. The platform relies on a stereo camera that captures left and right images processed by pre-trained DNNs to perform object perception, lane perception, and free space perception. An object tracker then estimates different metrics over the classified images, such as depth and radial distance, relative velocity, and azimuth and elevation angle. The system implements a 4-layer architecture deployed in each vehicle to guide cooperative autonomous navigation.

Systems that follow the *peer-to-peer first* sub-principle correspond to 20% of the reviewed articles. The rest of the articles (i.e., 80%) report systems that do not follow this sub-principle. Some of these are implemented as centralised architectures in the cloud [2, 7, 10, 13, 21, 48, 50, 57, 87, 110, 119, 127, 131]. Central entities in these deployments orchestrate the interactions between systems' components. Designers opt for such centralised control as it enables robust and flexible systems that are easy to scale and highly available. For example, Ali et al. [7] present ID-Viewer, a surveillance system for identifying infectious diseases in Pakistan. Data collection, analysis, and visualisation components process data from different areas of the country. A central server hosts these components to enable spatio-temporal analysis of the causes and evolution of infectious diseases. Several works also describe systems as federated architectures [8, 9, 15, 30, 35, 49, 56, 60, 79, 91, 111, 114, 116, 117, 126] without horizontal interactions between nodes at the same layer. Examples of these architectures are IoT deployments in different domains such as healthcare [35, 91, 111, 117], smart buildings [79, 116], farming [8], and predictive maintenance [9]. The architectures of these projects follow a layered pattern with vertical interaction between IoT devices, edge servers, and cloud servers. Designers use different layers to satisfy different system requirements. Lower layers (i.e., IoT devices) are in charge of data collection, middle layers (i.e., edge servers) are in charge of data preprocessing, filtering, and transmission, and upper layers (i.e., cloud servers) are in charge of data aggregation, decision making, visualisation, and user interaction. There are also self-contained systems deployed in single resource-constrained devices [14, 22, 44, 58, 63, 69, 89, 121]. These are embedded systems that combine hardware and software components for specific functions in environments that require fast reactions (e.g., neuroprosthetic devices [89]) or have limited connectivity with back-end servers (e.g., NASA's Mars rovers [121]).

5.3 Openness

Figure 5 shows to what extent practitioners adopt the **openness** principle. 22% of the articles rely on flexible architectures where new components can join autonomously. These new components join and cooperate with the existing ones to achieve the system's goals. Practitioners exploit this flexibility when systems must be scalable and highly available. Dynamic requirements need on-demand extension of storage and computing capabilities [37, 48, 61, 62, 68, 76, 124, 131, 144, 146]. Such flexibility causes security concerns as malicious entities can join the architecture. Designers must implement distributed security measures to address this risk. For example, Shan et al. [124] propose a decentralised architecture for Poligraph based on the BFT protocol, which adds servers and reviewers to the fake news detection process. Such an adaptation responds to the request number variability, enabling scalability, while parallelism enables low latency. A new node follows the BFT protocol to join the network. It sends messages to existing nodes to verify their identities, receives requests, and participates in the review consensus. The Sifter platform [144] is an online spam detection system for social networks. It uses an RNN for spam detection and DHTs to address the fast-changing nature of topics and events. Social data is stored by decentralised servers and grouped by topics using the DHT structure to enable fast retrieval. Each server responds to spam detection requests using its local neural network. New servers can join by following the DHT

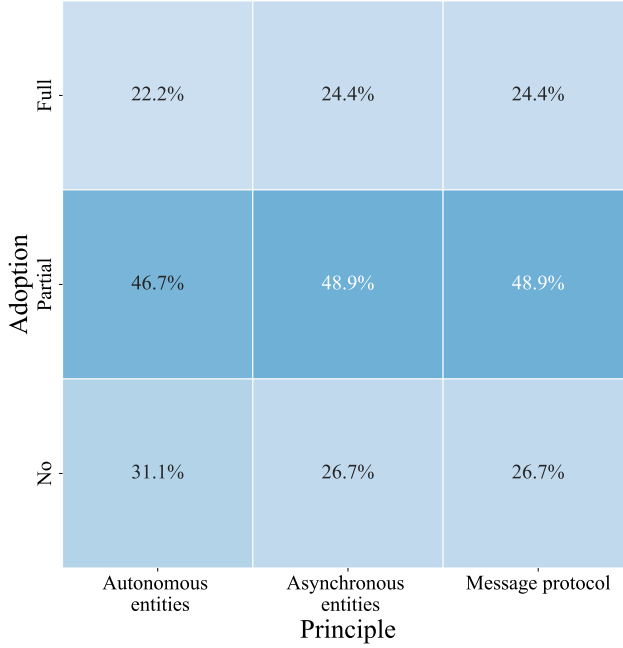


Fig. 5. Adoption of openness principles. Most reviewed systems make use of autonomous entities, but only a third use them asynchronously. More than 50% utilise message protocols.

protocol, which extends the storage capability of Sifter. A new node must contact an existing one, perform a lookup operation to find the closest nodes, and update these nodes' information and the network topology with its data. Local spam detection fits low latency requirements as requests are solved closer to end users. We observe that the technologies and protocols used to handle decentralised storage and peer-to-peer cooperation (Section 5.2) also support flexible architectures where entities can join autonomously. That is the case for protocols such as DHT [144], HDF5 [76], Spark [37], or BFT [124], and architectural patterns such as dataflow [62, 131], the observer [48], or publish/subscribe [61, 62, 68, 131, 146].

There are systems based on federated [8, 15, 30, 60, 114, 116, 117, 126, 147] and centralised [7, 9, 13, 35, 49, 56, 63, 79, 91, 110, 111, 127] architectures that partially (i.e., 46% of the articles) adopt the *autonomous entities* sub-principle. These architectures have mechanisms to add new sensing devices or users at the lowest layer, for instance, interoperable communication technologies that enable new actors to exchange messages with existing nodes. These technologies include Wi-Fi, Bluetooth, Zigbee, 5G, and so on. However, these systems are not flexible enough to include more storage or compute nodes at the edge or cloud layers. Authors leverage properties of autonomous components to implement and deploy flexible systems where new sensors expand systems' data collection capabilities and new users expand systems' service coverage. For example, Alonso et al. [8] use Fiware¹¹ as a middleware to manage devices that transmit data using the Zigbee wireless standard. The collected data support automated real-time decisions about the state of dairy cattle and the feed grain. Niu et al. [91] developed the SensorBox device that integrates different IoT sensors

¹¹Fiware: <https://www.fiware.org/>

to measure environmental variables. Supervised learning models are trained with such data to classify daily activities. Barachi et al. [13] deploy a crowdsensing platform based on DL to generate accident reports. Users are data consumers and collectors who can join the network by installing a mobile application. Calancea et al. [30] also require users to install an Android application to use the NLP-based platform that supports visually challenged people.

A third of the articles report ML-based systems whose components are not autonomous. Practitioners avoid autonomous components when system requirements do not demand dynamic changes to their components. These systems are self-contained embedded systems [14, 22, 44, 58, 69, 89, 121] or rely on cloud-based architectures [2, 10, 21, 50, 57, 87, 119]. The authors deploy embedded systems to perform specific tasks with limited resources. A change in the tasks of these systems implies their redesign. That is the case of the robots of the STRANDS project [58], specifically designed for navigation, human behaviour prediction, and activity recognition tasks. Similarly, Cabanes et al. [22] design an embedded system to detect parking events. It uses a single 3D sensor to collect the input data for object recognition. Cloud-based architectures rely on the cloud provider services to extend the storage or computing capabilities of the systems. This flexibility enables systems to satisfy availability and scalability requirements. For example, the Bighead framework [21] uses cloud-managed Kubernetes to support the development of data-driven solutions across the whole Airbnb organisation. Designers achieve such flexibility at the cost of the cloud service providers' fees.

Half of the articles report systems that partially adopt the *asynchronous entities* and the *message exchange protocol* sub-principles. Designers implement asynchronous communication because systems must listen to requests continuously or notify users at any time [13, 30, 56, 124, 127]. iAssistMe [30] relies on the pub/sub communication pattern to interact with visually challenged people and support their activities. The system uses RabbitMQ and the **Advanced Message Queuing Protocol (AMQP)** to handle data producers and consumers. Sharkeye [56] uses the **AWS Simple Notification Service (SNS)** to send messages warning people about the presence of sharks via smartwatches. Other authors implement systems that use asynchronous entities to listen to data sources (e.g., sensor devices) that can produce data at any time [2, 8, 9, 15, 35, 49, 57, 79, 87, 91, 110, 111, 114, 116, 117, 126, 147]. Bellocchio et al. [15] present the SEAL project to provide home automation based on ML for energy management and safety. SmartSEAL is the platform that manages IoT devices deployed in the buildings. It offers asynchronous and synchronous communication using pub/sub and client-server protocols. The ROS supports the pub/sub mechanism, where sensor nodes write to and read from a topic. They extend ROS with a central entity for device naming and registration. Synchronous API calls enable the consumption of these services. Alves et al. [9] propose an industrial system based on IoT nodes capable of acquiring and transmitting machine status parameters using the pub/sub mechanism (i.e., the MQTT protocol). IoT nodes post their readings in a shared database, which the system's components read asynchronously. Gallagher [49] uses synchronous and asynchronous mechanisms in the IntelliMaV platform. IntelliMaV is a system that applies ML to verify the performance of energy conservation measures in near real-time. The system is deployed in the cloud and follows a 3-layer architecture. The user tier is the access point for users through a web browser that presents an interface for model training and deployment. The cloud tier is a virtual private cloud and hosts the application infrastructure. This tier processes the application requests by running R code exposed through APIs. The site tier represents the pipelines that collect data from the industrial site. These pipelines push the collected data to the cloud storage asynchronously for further processing. The smart dairy tracer platform [8] uses different protocols to collect data from diverse sources. These protocols include Zigbee, LoRa, Wi-Fi, Bluetooth, and 3G, which support communication with sensors deployed on the crops and the barns used to feed the livestock, the cattle to monitor their health, the factories for the traceability of packaged products and energy monitoring, and the trucks that transport the

Table 7. Summary of Our Article Findings Against the Research Questions We Formulated in Section 4 for Each DOA Principle

DOA Principle	Adoption Extent (RQs To what extent?) (See Figures 3, 4, and 5 and Table 6)	Adoption Motivation (RQs Why?) (See Sections 5.1, 5.2, and 5.3)	Adoption in Practice (RQs How?) (See Sections 5.1, 5.2, and 5.3)
Data as a First Class Citizen	- All reviewed systems are <i>data-driven</i>. - Almost 75% of articles fully or partially adopt the <i>shared data model</i> sub-principle. - Less than half of the systems adopt <i>data coupling</i>.	ML-based systems use data-driven algorithms to satisfy their functional requirements. These systems must handle continuous data sources and historical data, usually large and dynamic. Data coupling fits these data management requirements better.	Practitioners use diverse data-driven algorithms to provide different functionalities. For example, DL models for fall detection, RL models for resource optimisation, Bayesian networks for rover monitoring, and so on. Designers implement <i>shared data models</i> and <i>data coupling</i> using databases (HDFS, HBase, etc.), streams (Apache Kafka, Spark Streaming, etc.), and message queues (e.g., RabbitMQ).
Prioritise Decentralisation	- Only 35% of works fully or partially adopt the <i>local data chunks</i> sub-principle. - Almost half of the reviewed articles fully or partially perform <i>local first</i> processing. - Only 20% of systems rely on components that prioritise <i>peer-to-peer</i> collaboration.	Practitioners implement decentralised or federated architectures because these enable systems to manage resources efficiently and be fault-tolerant thanks to data partitioning and replication. In addition, <i>local data chunks</i> improve system privacy and security as the data ownership does not change.	Practitioners rely on distributed storage and computing technologies and protocols to implement and manage decentralised or federated architectures. Examples of these technologies and protocols are Apache Kafka, HDFS, DHT, and BFT.
Openness	- Almost 70% of the reviewed systems rely on <i>autonomous entities</i>. - Almost 75% of articles rely on <i>asynchronous entities</i> that communicate using a <i>message protocol</i>. - Hybrid architectures prevail, combining static and autonomous entities based on synchronous and asynchronous communication protocols.	Dynamic environments like the ones we find in the real world require flexible architectures where new components (e.g., IoT sensors) can join at runtime. Practitioners adopt the <i>openness</i> principle to design and implement such architectures. <i>Autonomous entities</i> usually communicate in an <i>asynchronous</i> fashion using different <i>message exchange protocols</i> .	Practitioners use distributed computing (Apache Kafka, DHT, etc.) and communication technologies to implement <i>open</i> systems. Examples of communication technologies are Wi-Fi, Bluetooth, Zigbee, and 5G. Protocols like MQTT and RabbitMQ work on top of these technologies to enable <i>asynchronous communication</i> between <i>autonomous entities</i> .

The goal of this survey is to determine *to what extent and how researchers have adopted the DOA principles to implement current ML-based systems, and what are the motivations behind such adoption?*

dairy products. Sarabia et al. [117] propose a 3-layer fog-cloud architecture where new healthcare devices join using low-power wireless technologies such as Bluetooth, Zigbee, 6LoWPAN, and Wi-Fi. The Bluetooth protocol is also used by Quintero et al. [111] to include new devices to collect photoplethysmography data and heart rate monitoring.

The rest of the reviewed articles (i.e., 26.7%) report synchronous systems. These rely on traditional client/server architectures and communication patterns such as RPC or API calls. The authors opt for this design decision when systems are small, self-contained, and attend synchronous requests [14, 22, 44, 58, 60, 63, 69]. Hegemier et al. [60] propose a real-time danger avoidance system for robots. They deploy a neural network in an edge server, which receives prediction requests from robots in HTTP format. Requests include an image of the robot environment and its status information (e.g., damage report). The edge server responds synchronously with a classification of the image. Johnny et al. [63] propose a system for metastasis detection. They deploy a DL model in a Raspberry Pi, which performs image classification tasks. Users send their requests (i.e., images) to an embedded web server installed in the Pi. Components of larger systems also interact synchronously. These are usually deployed in the cloud and follow microservices architectures. They adopt layered and modular architectures that satisfy scalability and availability requirements [7, 10, 21, 50, 144]. For example, the AIDEx [10] system adopts an architecture to predict patients' risk of developing sepsis. The system encapsulates each functionality as a microservice. The system exposes microservices for preprocessing data, executing the prediction algorithm, storing the prediction outcomes, and visualising outcomes. The system orchestrates these components using API calls to offer end-to-end capabilities such as sepsis prediction.

5.4 Summary

Table 7 summarises our survey findings and answers the research question in Section 4. This section highlights the key findings and provides practical advice towards deploying Data-Oriented ML-based systems in the real world. We found that only one-fifth of the reviewed ML-based systems follow all or most DOA principles [37, 61, 62, 68, 76, 124, 146, 147]. We can thus conclude that the overall level of adoption of DOA principles in production-level ML systems is currently low. Despite

this low adoption, we found that designers follow DOA principles in the reviewed systems to satisfy requirements that are becoming increasingly common for data-driven systems. These systems address data requirements that demand managing big data from distributed and intensive sources, low-latency processing tasks, and efficient management of storage and computing resources. We observed that systems based on architectural patterns such as dataflow and publish/subscribe are more data-oriented and follow more DOA principles. These systems focus on data exchange between components (i.e., data first), which does not assume any coupling on the control flow level. **Practical advice:** Developers should focus first on the data while creating data-intensive systems, and consider operations as secondary. Dataflow and publish/subscribe patterns are well-suited for designing and implementing DOA systems.

We found that the **data as a first-class citizen** is the most adopted principle among the reviewed articles, mainly because of the type of requirements ML-based systems impose. These systems are naturally *data-driven*, implying that they must manage large and dynamic datasets. *Shared data models* and *data coupling* sub-principles fit this type of requirements better. Engineers adopt the **data as a first-class citizen** principle because it enables efficient interactions between system components. This design decision avoids data transmission between components as payloads of direct calls (e.g., API calls). Systems' components act as producers and consumers of data stored in *shared data models*. Such *data coupling* offers asynchronous interactions by design, which makes components autonomous and non-blocking for each other. *Data coupling* also addresses low-latency requirements and resource constraints because autonomous components can process large datasets in parallel. The nature of the data systems handled influences the selection of the *shared data models* that the systems' components nurture. *Data coupling* based on databases is appropriate for systems that work with historical data, while streams fit better systems that handle continuous data from dynamic sources. An engineer can design a system with one or more shared data models. This decision depends on how the system processes the data from inputs to outputs. **Practical advice:** *Data coupling* enables systems to address big data processing requirements in *data-driven* systems. Databases, streams, and message queues are examples of data mediums to consider when designing data-first systems. These data mediums play the role of *shared data models*, which practitioners can implement using technologies such as Apache Kafka, Spark Streaming, HDF5, RabbitMQ, or HBase.

Our results show that the **prioritise decentralisation** principle is less popular than the **data as a first-class citizen** principle because centralised cloud deployments prevail as the preferred choice when deploying systems nowadays. Practitioners have this strong preference due to the flexibility and maturity of cloud platforms. Decentralised architectures (e.g., edge computing) require communities to develop tools to facilitate their management for practitioners. We found that centralised architectures are the preferred design choice when deploying ML-based systems to the detriment of the **prioritise decentralisation** principle. Cloud platforms cope well with the most relevant requirements that ML-based systems have, as they provide flexibility, high availability, scalability, and mostly satisfy current latency requirements. However, it is natural to expect these requirements to become more critical as data becomes more available, users expect faster responses, and applications become more complex (digital twins, augmented reality, etc.). Further research on decentralised architectures is necessary to enable the benefits of decentralisation while making it a feasible option for deployment. Reviewed works relying on edge computing already leverage federated architectures with increasingly powerful edge servers that offer *local data storage and processing*. However, the *peer-to-peer* interaction between nodes is missing in most cases. Back-end cloud servers still play the central storage and processing roles. A collaboration between nodes at lower layers in edge architectures can enable more sustainable systems by exploiting the storage and computing power of everyday devices [74]. **Practical advice:** The absence of a central orchestrator in favour of direct communication between any two nodes of the system

is a straightforward way to move to decentralisation. The authors of the reviewed articles used distributed storage technologies (Apache Kafka, HDFS, etc.) and distributed protocols (e.g., DHTs and BFT) to implement decentralised solutions.

Systems that collect data from distributed devices (e.g., sensors) are usually **open**. These systems use communication technologies such as Wi-Fi, Bluetooth, Zigbee, and 5G to add new sensors and data sources. Architectures are more closed and static at upper levels, whereas software components are static entities that rely on synchronous communication via RPC or API calls. We observe a strong correlation between data coupling and openness. Reading from and writing to data mediums is an *asynchronous communication* process where systems' components operate as data consumers and producers. This process requires components to utilise *message exchange protocols* that describe how and when to read and write data. Such protocols also enable seamless and flexible architectures where components can join or leave autonomously. **Practical advice:** The use of message exchange protocols and data coupling results in systems that are open and flexible by design. It enables data availability and horizontal scalability as systems add resources on demand. Communication protocols such as MQTT and RabbitMQ are well-known tools for building open systems.

5.5 Threats to validity

In this section, we discuss threats to the validity and limitations of our work.

Internal validity. We used a multi-stage selection process to find articles for the survey. We have followed an established methodology (SLR, [72, 73]). However, this approach can have drawbacks.

There might be missed terms in the search queries that we have run against digital libraries. To mitigate this risk, we have iteratively improved our search procedure multiple times based on the analysis of the search output.

The search functionalities between different databases used in our automatic search differ. We addressed this risk by including as many publicly available library APIs as possible, even if their content overlaps. Our tool is publicly available and is easy to extend to include new sources in the future to ensure quality and transparency.

The number of retrieved works required to automate the filtering process. We used the state-of-the-art algorithm Lbl2Vec to implement the automatic filter. However, this semantic filter could exclude relevant works. We tested the Lbl2Vec algorithm under different configurations to determine the best possible result. The Lbl2Vec algorithm requires defining classes for categorising articles as included and excluded. We explore different configurations for the classes and validate the outputs against the desired research works we expected to include. Furthermore, we used the snowballing step in the methodology to identify relevant articles we could have missed.

External validity. We can generalise the conclusions in our study beyond as general advice for practitioners. Here, we discuss potential threats to such generalisation and how we addressed them.

The scientific literature does not describe many ML deployments. Some deployments are presented as blog posts, while others are not published anywhere. We did not include such projects in our study and dismissed some published reports because they omitted information about their software architecture from the article. Nevertheless, our survey covers a wide range of domains, including ML deployments across fields from healthcare to autonomous driving (see Table 3). We have also taken care not to impose unjustified limits on the search procedure to improve coverage.

The analysis and conclusions of this article can be prone to personal biases as they rely on the authors' expertise. Following the advice by Kitchenham et al. [71], the authors of this survey reviewed each other's study selection, analysis and conclusions. To further mitigate this risk, we sought feedback on our work, presenting intermediate results internally to our research group and externally at academic events.

Table 8. Survey Findings, Open Challenges, and Possible Actions to Address Them

Survey Findings	Open Challenge	Research Directions
A few articles fully adopt the DOA principles. (RQ1, RQ2, RQ3)	The DOA style is not fully integrated with other ML areas and the practitioners' workflow.	The DOA style offers a range of desirable properties for multiple software systems. Integrating the DOA style within other AI and ML areas is an interesting and open research area. Alternative research paths in this area are: - Develop DOA-based systems as enablers of ML pipelines (emulators, Bayesian optimisation, etc.). - Explore the intersection between decentralised DOA-based systems and fields like edge computing, federated learning, self-adaptive systems, and causal inference. - Case studies to define the role of DOA-based systems in data management and governance. - Study how DOA principles can support LLMs in collecting data for fine-tuning or enriching prompting.
Only one of the reviewed articles uses a <i>shared data model</i> and <i>data coupling</i> with monitoring purposes. (RQ1)	Current systems do not exploit the DOA style monitoring properties.	One of the key features that <i>shared data models</i> and <i>data coupling</i> enable is the capability of systems self-monitoring. Research paths in this direction are: - Develop frameworks to enable statistical emulators to consume data that systems expose in <i>shared data models</i>. - Study and define how to apply the emulators' mathematical concept in the software engineering domain. - Exploit data exposed in DOAs to feed LLMs or transformers that can inform users about system states. - Case studies to improve our understanding of the relationships between DOAs, surrogate models, and LLMs.
Practitioners prefer centralised cloud deployments based on client/server or microservices over <i>decentralised</i> architectures when deploying the current ML-based systems. (RQ2)	Communities still miss tools and practices around the DOA style of system architecture.	The lack of maturity of the DOA style negatively impacts its adoption in different domains. Practitioners prefer centralised cloud deployments, relying on client/server or microservices architectures. We must research and develop knowledge, practices, and tools around the DOA style. Alternative research paths in this direction are: - Build a knowledge base for DOA similar to the one communities have developed for SOA. - Case studies to research and quantify the limits of the DOA architectural style. - Research on the impact of the DOA style on the system's lifecycle (i.e., design, development, deployment, and decommissioning).
Systems architectures are flexible at lower layers to add new sensors or attend to users dynamically. However, this <i>openness</i> is absent in the upper layers mainly because systems' requirements do not demand dynamic architectures at these levels. (RQ3)	The fast pace of technological evolution is exacerbating systems requirements towards more flexible architectures, which threaten data security and privacy.	Future software systems will have exacerbated data requirements, demanding flexible architectures where <i>openness</i> plays a crucial role in creating decentralised deployments and exploiting the computing power of everyday devices. These flexible and open architectures generate privacy and security concerns that communities must mitigate. Research paths in this direction are: - Explore and exploit different approaches from the security community to improve security and privacy aspects of the DOA style. - Implement and evaluate homomorphic encryption and zero-knowledge proof in the DOA context. - Exploit decentralisation to deploy systems that store and process data locally (i.e., end users' devices). - Develop the DOA style and appropriate tools for secure deployment in resource-constrained devices.

6 Open Research Challenges

We observe that while the DOA principles offer the desired properties that enable ML-based systems to achieve demanding requirements at deployment, their adoption rates were mostly low. More research efforts are needed to advance the community's understanding of DOA, its strengths and weaknesses, why and how to build DOA systems, and the advantages of the capabilities DOA offers. Table 8 summarises the identified DOA limitations from our survey. The rest of this section discusses them and formulates future research directions accordingly.

6.1 Interplay with Other Areas

The community needs additional efforts to research the interplay between the DOA style and other areas of computer science. In this section, we suggest some of the research ideas that DOA can enable.

Shared data models offer opportunities for new practices around emulating individual systems' components. Such emulation ability can lead to efficient systems' end-to-end optimisation [3, 145]. Such optimisation is particularly pertinent in cases where the system has to satisfy multiple competing requirements [12], in which multi-objective Bayesian optimisation techniques can be applicable [103]. DOA can also accelerate advances in the area of Edge Computing [28, 125, 132], Federated Learning [20] and self-adaptive systems [25, 52], all of which stand to benefit from a distributed shared data model as well as autonomous open environments. The ability to discover and inspect explicit flows of data within DOA systems has been highlighted as a key feature required for data governance [5, 31, 123, 128], meaning DOA can play a role in developing AI systems that comply with existing and upcoming legislatures, such as GDPR, EU AI Act or **Equal Credit Opportunity Act (ECOA)**. Similarly, access to a complete dataflow graph means engineers can leverage causal inference techniques [101, 102].

The field of **Natural Language Processing (NLP)** has been experiencing rapid growth, with an increasing focus on LLMs. Models such as GPT-4 [94], Llama [134] or Codex [33] have shown remarkable capabilities in understanding and generating human-like text or computer code. These massive models require significant computational and storage resources, making software infrastructure a key determinant of their success. Given LLMs' dependency on the quantity and quality of data, DOAs bring benefits to building efficient and robust infrastructure for them. An open research direction is to study the use of DOA principles for building LLMs and how DOA can improve their infrastructure. LLMs depend on the quality of the training data, the quality of the prompts, and the context users provide when asking questions. DOA-based systems can facilitate training and contextual data collection to fine-tune and interact with LLMs. The realisation of this vision enables promising opportunities for software generation and maintenance [113] and to create novel interfaces between humans and machines to keep users in control despite the systems' complexity [27].

6.2 Systems Monitoring and Shadow Systems

DOA proposes that systems' components interact through data mediums. Shared data models store systems' historic states that are fully available. This data availability should facilitate systems' monitoring tasks in contrast to design paradigms where interfaces hide data (e.g., microservices). However, in practice, we found that only one of the forty-five reviewed articles uses the shared data model for monitoring. The robotic platform proposed by Hawes et al. [58] monitors historical database records of the robot's status and its interactions with the environment. This limitation emerges because communities lack tools for monitoring DOA-based systems. New practices and frameworks for monitoring DOA systems are needed. These practices can leverage the systems' data availability offered by the DOA style.

One possible approach that fits the key features of DOA is a network of statistical emulators (i.e., shadow systems). A statistical emulator is a probabilistic surrogate model of a given process that allows uncertainty to be quantified to inform decision-making [104]. By exposing all intermediate data interactions within the system, DOA can enable the creation and maintenance of a separate emulator for each system component. A network of such emulators acts as a shadow system, which measures the gap in behaviour between the real-world deployment and the designed software system. Such measurement includes monitoring, identifying fluctuations in data streams, and estimating the effects of potential changes. Two research efforts are required to move the idea of a shadow system beyond a mere concept. First, developing the DOA paradigm to identify the most efficient approaches and practices around the automatic fitting of surrogate models to software components. Existing work focuses on auto-tuning of system parameters [6, 38] and has limited scalability. Additional use cases should implement and test shadow emulators as

monitoring and explainability tools for software, to explore scalable ways of automatically building surrogate models of systems' components. Second, innovation in the mathematical composition of individual emulators is required to build networks that can efficiently propagate uncertainty between components [39]. While there is prior work on uncertainty propagation in software [83], it is still uncommon to see interfaces and APIs that provide access, or otherwise handle, input or output uncertainty.

6.3 DOA Concept Maturity and Tooling

Previous sections show practitioners rely on several tools and frameworks while building data-driven systems. Databases, streams, message queues and file systems were all used as data communication mediums. At the same time, there is no reference that engineers could use while deciding which technology to use in their system. The adoption of DOA is now limited because the concept itself lacks maturity. Practitioners prefer centralised cloud deployments based on traditional client/server and microservices architectures. DOA can benefit from having a technical stack taxonomy so developers can have a complete list of necessary abstractions in a DOA system and a list of tools to choose for each abstraction. Such taxonomies already exist for other areas of engineering practice, such as MLOps¹² or microservices [51].

A question related to the choice of tools and frameworks is the operational maintenance of DOA systems. Over the years, the software engineering community accumulated a vast amount of knowledge on how to run SOA systems in production: what metrics to monitor, how to scale horizontally and vertically, how to mitigate and troubleshoot performance issues [16, 40, 78, 129]. A similar knowledge base about DOA systems is necessary to accumulate experience running DOA software. Nevertheless, researchers can study configurations of DOA setups, their performance, and critical metrics. The goal of such studies must be to quantify the impact of the DOA style on the systems' life-cycle.

6.4 Systems Security and Privacy

The DOA style encourages practitioners to implement open systems where autonomous entities can freely access shared data models [82, 107, 141]. These data models can contain sensitive data, which raises questions regarding security and privacy [135]. Malicious entities can access and modify systems' data and behaviour. Designers must restrict data access and system components to a specific set of users, considering different policies depending on the application. Decentralised deployments can mitigate security and privacy threats by storing and processing data in devices closer to end users. For example, the crowd management system proposed by Santana et al. [116] anonymises Wi-Fi frames at the edge to protect people's identities. The authors use these frames to train a learning model for crowdsensing estimation. Despite these efforts, managing authentication, permissions, and encryption in open environments is challenging.

Different research efforts from the security community can support DOA open setups to address security and privacy challenges. Homomorphic encryption [47] is an interesting direction for performing decentralised computations on encrypted data without providing the decryption key to the participant nodes. For example, a payment system might inquire about the transaction's validity without having access to the underlying data (e.g., bank account number). Deployment of zero-knowledge proof [45] and homomorphic encryption is taking place in the industry [18]. This technology address privacy issues in decentralised networks, but the field is still in its infancy. Algorithms developed are often computationally expensive, so further research is needed to make them practical for constrained devices. These research efforts must align with decentralised deploy-

¹²AI Infrastructure Alliance project: <https://ai-infrastructure.org/>

ments, as this DOA principle benefits privacy and security attributes by exploiting local storage and computing. In addition to these technical advances, security and privacy issues require authorities to develop novel initiatives and policy frameworks to keep up with advances in technology [84].

7 Conclusions

This article surveys the extent to which, why, and how practitioners have adopted DOA to design, implement, and deploy ML-based systems in the real world. We discuss the DOA principles based on the existing literature and explain how these can support practitioners in addressing the challenges that emerge from deploying ML models as part of larger systems. The works analysed in this survey were selected using a semi-automatic process to facilitate this study's reproducibility and evolution. We developed this process based on a well-known methodology for literature reviews in the software engineering domain. The code, raw, and preprocessed data generated in this process are available online.

Our survey shows that a few systems fully adopt the DOA principles, with most reviewed works demonstrating partial adoption. We observe that systems that follow the DOA principles are mostly data-intensive. They address requirements such as big data management, low-latency real-time processing, efficient resource management, security, and privacy. We show the protocols and tools these systems use for adopting the principles and distil recommendations for practitioners to build their systems following DOA. We finish by formulating open challenges that can improve the community's understanding and application of DOA. We expect this work to increase community awareness of DOA and ignite interest in research, development, and adoption of this promising architectural style for ML-based systems.

Acknowledgments

We thank Oluwatomisin Dada, Eric Meissner, Markus Kaiser, and Peter Ochieng for helpful and insightful discussions on this article.

References

- [1] Mike Acton. 2014. Data-oriented design and C++. In *CppCon 2014*, CppCon.
- [2] Alekh Agarwal, Sarah Bird, Markus Cozowicz, Luong Hoang, John Langford, Stephen Lee, Jiaji Li, Dan Melamed, Gal Oshri, Oswaldo Ribas, Siddhartha Sen, and Alex Slivkins. 2016. Making contextual decisions with low technical Debt. DOI : <https://doi.org/10.48550/ARXIV.1606.03966>
- [3] Virginia Aglietti, Xiaoyu Lu, Andrei Paleyes, and Javier González. 2020. Causal bayesian optimization. In *Proceedings of the Artificial Intelligence and Statistics*. PMLR.
- [4] Hussain Akbar, Muhammad Zubair, and Muhammad Shairoze Malik. 2023. The security issues and challenges in cloud computing. *International Journal for Electronic Crime Investigation* 7, 1 (2023), 13–32.
- [5] Sherif Akoush, Andrei Paleyes, Arnaud Van Looveren, and Clive Cox. 2022. Desiderata for next generation of ML model serving. In *Proceedings of the Workshop on Challenges in Deploying and Monitoring Machine Learning Systems, NeurIPS*.
- [6] Sami Alabed and Eiko Yoneki. 2022. BoGraph: Structured bayesian optimization from logs for expensive systems with many parameters. In *Proceedings of the 2nd European Workshop on Machine Learning and Systems*.
- [7] M. A. Ali, Z. Ahsan, M. Amin, S. Latif, A. Ayyaz, and M. N. Ayyaz. 2016. ID-Viewer: a visual analytics architecture for infectious diseases surveillance and response management in Pakistan. *Public Health* 134 (2016), 72–85. DOI : <https://doi.org/10.1016/j.puhe.2016.01.006>
- [8] Ricardo S. Alonso, Inés Sittón-Candanedo, Óscar García, Javier Prieto, and Sara Rodríguez-González. 2020. An intelligent Edge-IoT platform for monitoring livestock and crops in a dairy farming scenario. *Ad Hoc Networks* 98 (2020), 102047. DOI : <https://doi.org/10.1016/j.adhoc.2019.102047>
- [9] Filipe Alves, Hasmik Badikyan, H. J. António Moreira, João Azevedo, Pedro Miguel Moreira, Luís Romero, and Paulo Leitão. 2020. Deployment of a smart and predictive maintenance system in an industrial case study. In *Proceedings of the 2020 IEEE 29th International Symposium on Industrial Electronics (ISIE)*. 493–498. DOI : <https://doi.org/10.1109/ISIE45063.2020.9152441>

- [10] Fatemeh Amrollahi, Supreeth Prajwal Shashikumar, Pradeeban Kathiravelu, Ashish Sharma, and Shamim Nemati. 2020. AIDEx - An open-source platform for real-time forecasting sepsis and a case study on taking ML algorithms to production. In *Proceedings of the 2020 42nd Annual International Conference of the IEEE Engineering in Medicine and Biology Society (EMBC)*. 5610–5614. DOI : <https://doi.org/10.1109/EMBC44109.2020.9175947>
- [11] Mauricio Aniche, Joseph Yoder, and Fabio Kon. 2019. Current challenges in practical object-oriented software design. In *Proceedings of the 2019 IEEE/ACM 41st International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER)*. IEEE, 113–116.
- [12] Brendan Avent, Javier González, Tom Diethe, Andrei Paleyes, and Borja Balle. 2020. Automatic discovery of privacyutility pareto fronts. *Proceedings on Privacy Enhancing Technologies* 2020, 4 (October 2020), 5–23. DOI : <https://doi.org/10.2478/popets-2020-0060>
- [13] May El Barachi, Faouzi Kamoun, Jannatul Ferdaos, Mouna Makni, and Imed Amri. 2020. An artificial intelligence based crowdsensing solution for on-demand accident scene monitoring. *Procedia Computer Science* 170 (2020), 303–310. DOI : <https://doi.org/10.1016/j.procs.2020.03.044>
- [14] Sebastian P. Bayerl, Tommaso Frassetto, Patrick Jauernig, Korbinian Riedhammer, Ahmad-Reza Sadeghi, Thomas Schneider, Emmanuel Stapf, and Christian Weinert. 2020. Offline model guard: Secure and private ML on mobile devices. In *Proceedings of the 2020 Design, Automation and Test in Europe Conference and Exhibition (DATE)*. 460–465. DOI : <https://doi.org/10.23919/DATE48585.2020.9116560>
- [15] Enrico Bellocchio, Gabriele Costante, Silvia Cascianelli, Paolo Valigi, and Thomas A. Ciarfuglia. 2016. SmartSEAL: A ROS based home automation framework for heterogeneous devices interconnection in smart buildings. In *Proceedings of the 2016 IEEE International Smart Cities Conference (ISC2)*. 1–6. DOI : <https://doi.org/10.1109/ISC2.2016.7580798>
- [16] Betsy Beyer, Chris Jones, Jennifer Petoff, and Niall Richard Murphy. 2016. *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, Inc..
- [17] Bernd Bischl, Martin Binder, Michel Lang, Tobias Pielok, Jakob Richter, Stefan Coors, Janek Thomas, Theresa Ullmann, Marc Becker, Anne-Laure Boulesteix, Difan Deng, and Marius Lindauer. 2023. Hyperparameter optimization: Foundations, algorithms, best practices, and open challenges. *WIREs Data Mining and Knowledge Discovery* 13, 2 (2023), e1484. DOI : <https://doi.org/10.1002/widm.1484>
- [18] Manuel Blum, Paul Feldman, and Silvio Micali. 1919. Non-interactive zero-knowledge and its applications. In *Providing of the Sound Foundations for Cryptography: On the Work of Shafi Goldwasser and Silvio Micali*. 329–349.
- [19] Jeannette Bohg, Antonio Morales, Tamim Asfour, and Danica Kragic. 2013. Data-driven grasp synthesis—a survey. *IEEE Transactions on Robotics* 30, 2 (2013), 289–309.
- [20] Keith Bonawitz, Hubert Eichner, Wolfgang Grieskamp, Dzmitry Huba, Alex Ingerman, Vladimir Ivanov, Chloé Kiddon, Jakub Konečný, Stefano Mazzocchi, Brendan McMahan, et al. 2019. Towards federated learning at scale: System design. In *Proceedings of the Machine Learning and Systems*. A. Talwalkar, V. Smith, and M. Zaharia (Eds.), Vol. 1, 374–388. Retrieved from <https://proceedings.mlsys.org/paper/2019/file/bd686fd640be98efaae0091fa301e613-Paper.pdf>
- [21] Eli Brumbaugh, Mani Bhushan, Andrew Cheong, Michelle Gu-Qian Du, Jeff Feng, Nick Handel, Andrew Hoh, Jack Hone, Brad Hunter, Atul Kale, et al. 2019. Bighead: A framework-agnostic, end-to-end machine learning platform. In *Proceedings of the 2019 IEEE International Conference on Data Science and Advanced Analytics (DSAA)*. 551–560. DOI : <https://doi.org/10.1109/DSAA.2019.00070>
- [22] Q. Cabanes, B. Senouci, and A. Ramdane-Cherif. 2019. A complete multi-CPU/FPGA-based design and prototyping methodology for autonomous vehicles: Multiple object detection and recognition case study. In *Proceedings of the 2019 International Conference on Artificial Intelligence in Information and Communication (ICAIIIC)*. 158–163. DOI : <https://doi.org/10.1109/ICAIIIC.2019.8669047>
- [23] Christian Cabrera, Viviana Bastidas, Jennifer Schooling, and Neil D. Lawrence. 2025. The systems engineering approach in times of large language models. In *Proceedings of the 58th Hawaii International Conference on System Sciences*. DOI : <https://doi.org/10.24251/HICSS.2025.666>
- [24] Christian Cabrera and Siobhán Clarke. 2021. A reinforcement learning-based service model for the internet of things. In *Service-Oriented Computing*. Hakim Hacid, Odej Kao, Massimo Mecella, Naouel Moha, and Hye-young Paik (Eds.), Springer International Publishing, Cham, 790–799.
- [25] Christian Cabrera and Siobhán Clarke. 2022. A self-adaptive service discovery model for smart cities. *IEEE Transactions on Services Computing* 15, 1 (2022), 386–399. DOI : <https://doi.org/10.1109/TSC.2019.2944356>
- [26] Christian Cabrera, Fan Li, Vivek Nallur, Andrei Palade, M. A. Razzaque, Gary White, and Siobhán Clarke. 2017. Implementing heterogeneous, autonomous, and resilient services in IoT: An experience report. In *Proceedings of the 2017 IEEE 18th International Symposium on A World of Wireless, Mobile and Multimedia Networks (WoWMoM)*. 1–6. DOI : <https://doi.org/10.1109/WoWMoM.2017.7974341>
- [27] Christian Cabrera, Andrei Paleyes, and Neil David Lawrence. 2024. Self-sustaining Software Systems (S4): Towards improved interpretability and adaptation. In *Proceedings of the 1st International Workshop on New Trends in Software Architecture (SATrends '24)*. Association for Computing Machinery, New York, NY, USA, 5–9. DOI : <https://doi.org/10.1145/3643657.3643910>

- [28] Christian Cabrera, Sergej Svorobej, Andrei Palade, Aqeel Kazmi, and Siobhán Clarke. 2023. MAACO: A dynamic service placement model for smart cities. *IEEE Transactions on Services Computing* 16, 1 (2023), 424–437. DOI : <https://doi.org/10.1109/TSC.2022.3143029>
- [29] Qiong Cai, Hao Wang, Zhenmin Li, and Xiao Liu. 2019. A survey on multimodal data-driven smart healthcare systems: Approaches and applications. *IEEE Access* 7 (2019), 133583–133599. DOI : <https://doi.org/10.1109/ACCESS.2019.2941419>
- [30] Cristina Georgiana Calancea, Camelia-Maria Miluț, Lenuța Alboae, and Adrian Ifțene. 2019. iAssistMe - Adaptable Assistant for Persons with Eye Disabilities. *Procedia Computer Science* 159 (2019), 145–154. DOI : <https://doi.org/10.1016/j.procs.2019.09.169>
- [31] Lucian Carata, Sherif Akoush, Nikilesh Balakrishnan, Thomas Bytheway, Ripduman Sohan, Margo Seltzer, and Andy Hopper. 2014. A primer on provenance. *Communications of the ACM* 57, 5 (2014), 52–60.
- [32] Chengliang Chai, Jiayi Wang, Yuyu Luo, Zeping Niu, and Guoliang Li. 2022. Data management for machine learning: A survey. *IEEE Transactions on Knowledge and Data Engineering* 35, 5 (2022), 4646–4667.
- [33] Mark Chen, Jerry Twarek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating large language models trained on code. *ArXiv abs/2107.03374*. Retrieved from <http://arxiv.org/abs/2107.03374>
- [34] Jennifer Cobbe, Michael Veale, and Jatinder Singh. 2023. Understanding accountability in algorithmic supply chains. In *Proceedings of the 2023 ACM Conference on Fairness, Accountability, and Transparency (FACt '23)*. Association for Computing Machinery, New York, NY, USA, 1186–1197. DOI : <https://doi.org/10.1145/3593013.3594073>
- [35] Bryan Conroy, Ikaro Silva, Golbarg Mehraei, Robert Damiano, Brian Gross, Emmanuele Salvati, Ting Feng, Jeffrey Schneider, Niels Olson, Anne G. Rizzo, et al. 2022. Real-time infection prediction with wearable physiological monitoring and AI to aid military workforce readiness during COVID-19. *Scientific Reports* 12, 1 (2022), 1–12.
- [36] David E. Culler. 1986. Dataflow architectures. *Annual Review of Computer Science* 1, 1 (1986), 225–253.
- [37] Jason Jinquan Dai, Yiheng Wang, Xin Qiu, Ding Ding, Yao Zhang, Yanzhang Wang, Xianyan Jia, Cherry Li Zhang, Yan Wan, Zhichao Li, et al. 2019. BigDL: A distributed deep learning framework for big data. In *Proceedings of the ACM Symposium on Cloud Computing (SoCC '19)*. Association for Computing Machinery, New York, NY, USA, 50–60. DOI : <https://doi.org/10.1145/3357223.3362707>
- [38] Valentin Dalibard, Michael Schaarschmidt, and Eiko Yoneki. 2017. BOAT: Building auto-tuners with structured Bayesian optimization. In *Proceedings of the 26th International Conference on World Wide Web*. 479–488.
- [39] Andreas Damianou and Neil D. Lawrence. 2013. Deep gaussian processes. In *Proceedings of the Artificial Intelligence and Statistics*. PMLR, 207–215.
- [40] Goran Delac, Marin Silic, and Sinisa Sribljic. 2012. Reliability modeling for SOA systems. In *2012 Proceedings of the 35th International Convention MIPRO*. IEEE, 847–852.
- [41] Filip Karlo Došilović, Mario Brčić, and Nikica Hlupić. 2018. Explainable artificial intelligence: A survey. In *Proceedings of the 2018 41st International Convention on Information and Communication Technology, Electronics and Microelectronics (MIPRO)*. IEEE, 0210–0215.
- [42] Tugba Akinci D'Antonoli, Arnaldo Stanzione, Christian Bluethgen, Federica Vernuccio, Lorenzo Ugga, Michail E. Klontzas, Renato Cuocolo, Roberto Cannella, and Burak Koçak. 2024. Large language models in radiology: Fundamentals, applications, ethical considerations, risks, and future directions. *Diagnostic and Interventional Radiology* 30, 2 (2024), 80.
- [43] Hugo Jair Escalante. 2021. Automated Machine Learning—A brief review at the end of the early years. In *Automated Design of Machine Learning and Search Algorithms*, Nelishia Pillay and Rong Qu (Eds.). Springer International Publishing, Cham, 11–28. DOI : https://doi.org/10.1007/978-3-030-72069-8_2
- [44] Joao Diogo Falcao, Prabh Simran S. Baweja, Yi Wang, Akkarit Sangpetch, Hae Young Noh, Orathai Sangpetch, and Pei Zhang. 2021. PIWIMS: Physics Informed Warehouse Inventory Monitory via Synthetic data generation. In *Adjunct Proceedings of the 2021 ACM International Joint Conference on Pervasive and Ubiquitous Computing and Proceedings of the 2021 ACM International Symposium on Wearable Computers (UbiComp '21)*. Association for Computing Machinery, New York, NY, USA, 613–618. DOI : <https://doi.org/10.1145/3460418.3480415>
- [45] Uriel Fiege, Amos Fiat, and Adi Shamir. 1987. Zero knowledge proofs of identity. In *Proceedings of the 19th Annual ACM Symposium on Theory of Computing*. 210–217.
- [46] Kathi Fisler, Shriram Krishnamurthi, Benjamin S. Lerner, and Joe Gibbs Politz. 2021. *A Data-Centric Introduction to Computing*. <https://dcic-world.org/2025-08-27/index.html>

- [47] Caroline Fontaine and Fabien Galand. 2007. A survey of homomorphic encryption for nonspecialists. *EURASIP Journal on Information Security* 2007, 1 (2007), 013801.
- [48] Stan Franklin, Tamas Madl, Sidney D'Mello, and Javier Snaider. 2014. LIDA: A systems-level architecture for cognition, emotion, and learning. *IEEE Transactions on Autonomous Mental Development* 6, 1 (2014), 19–41. DOI : <https://doi.org/10.1109/TAMD.2013.2277589>
- [49] Colm V. Gallagher, Kevin Leahy, Peter O'Donovan, Ken Bruton, and Dominic T. J. O'Sullivan. 2019. IntelliMaV: A cloud computing measurement and verification 2.0 application for automated, near real-time energy savings quantification and performance deviation detection. *Energy and Buildings* 185 (2019), 26–38. DOI : <https://doi.org/10.1016/j.enbuild.2018.12.034>
- [50] Guanyu Gao, Yonggang Wen, and Cedric Westphal. 2016. Resource provisioning and profit maximization for transcoding in Information Centric Networking. In *Proceedings of the 2016 IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*. 97–102. DOI : <https://doi.org/10.1109/INFCOMW.2016.7562053>
- [51] Martin Garriga. 2018. Towards a taxonomy of microservices architectures. In *Proceedings of the Software Engineering and Formal Methods: SEFM 2017 Collocated Workshops: DataMod, FAACS, MSE, CoSim-CPS, and FOCLASA, Trento, Italy, September 4-5, 2017, Revised Selected Papers 15*. Springer, 203–218.
- [52] Simos Gerasimou, Thomas Vogel, and Ada Diaconescu. 2019. Software engineering for intelligent and autonomous systems: report from the GI dagstuhl seminar 18343. DOI : <https://doi.org/10.48550/ARXIV.1904.01518>
- [53] Görkem Giray. 2021. A software engineering perspective on engineering machine learning systems: State of the art and challenges. *Journal of Systems and Software* 180 (2021), 111031. DOI : <https://doi.org/10.1016/j.jss.2021.111031>
- [54] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. 2017. Deep learning (Adaptive Computation and Machine Learning series). The MIT Press.
- [55] Ian J. Goodfellow, Oriol Vinyals, and Andrew M. Saxe. 2015. Qualitatively characterizing neural network optimization problems. *arXiv*. Retrieved from <http://arxiv.org/abs/1412.6544>
- [56] Robert Gorkin, Kye Adams, Matthew J. Berryman, Sam Aubin, Wanqing Li, Andrew R. Davis, and Johan Barthelemy. 2020. Sharkeye: Real-time autonomous personal shark alerting via aerial surveillance. *Drones* 4, 2 (2020). DOI : <https://doi.org/10.3390/drones4020018>
- [57] Hassan Habibi Gharakheili, Minzhao Lyu, Yu Wang, Himal Kumar, and Vijay Sivaraman. 2019. iTeleScope: Softwarized network middle-box for real-time video telemetry and classification. *IEEE Transactions on Network and Service Management* 16, 3 (2019), 1071–1085. DOI : <https://doi.org/10.1109/TNSM.2019.2929511>
- [58] Nick Hawes, Christopher Burbridge, Ferdian Jovan, Lars Kunze, Bruno Lacerda, Lenka Mudrova, Jay Young, Jeremy Wyatt, Denise Hebesberger, Tobias Kortner, et al. 2017. The STRANDS project: Long-term autonomy in everyday environments. *IEEE Robotics and Automation Magazine* 24, 3 (2017), 146–156. DOI : <https://doi.org/10.1109/MRA.2016.2636359>
- [59] Petra Heck. 2024. What about the data? a mapping study on data engineering for AI systems. In *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering-Software Engineering for AI*. 43–52.
- [60] Henri Hegemier and Jaimie Kelley. 2021. Image classification with knowledge-based systems on the edge for real-time danger avoidance in robots. In *Proceedings of the 2021 IEEE World AI IoT Congress (AllIoT)*. 0360–0365. DOI : <https://doi.org/10.1109/AIIoT52608.2021.9454183>
- [61] Ricardo Dintén Herrero and Marta Zorrilla. 2022. An I4.0 data intensive platform suitable for the deployment of machine learning models: a predictive maintenance service case study. *Procedia Computer Science* 200 (2022), 1014–1023. DOI : <https://doi.org/10.1016/j.procs.2022.01.300>
- [62] Junchen Jiang, Shijie Sun, Vyas Sekar, and Hui Zhang. 2017. Pytheas: Enabling data-driven quality of experience optimization using group-based exploration-exploitation. In *Proceedings of the 14th USENIX Conference on Networked Systems Design and Implementation (NSDI'17)*. USENIX Association, USA, 393–406.
- [63] Anil Johny and K. N. Madhusoodanan. 2021. Edge computing using embedded webserver with mobile device for diagnosis and prediction of metastasis in histopathological images. *International Journal of Computational Intelligence Systems* 14, 1 (December 2021), 194. DOI : <https://doi.org/10.1007/s44196-021-00040-x>
- [64] Praveen Joshi, Mohammed Hasanuzzaman, Chandra Thapa, Haithem Afli, and Ted Scully. 2023. Enabling All In-edge deep learning: A literature review. *IEEE Access* 11 (2023), 3431–3460. DOI : <https://doi.org/10.1109/ACCESS.2023.3234761>
- [65] Rajive Joshi. 2007. Data-oriented architecture: A loosely-coupled real-time SOA. *Real-Time Innovations, Inc* (August 2007), 1–54. Retrieved from <https://community.rti.com/archive/data-oriented-architecture-loosely-coupled-real-time-soa>
- [66] J. Stephen Judd. 1990. *Neural Network Design and the Complexity of Learning*. MIT Press.
- [67] Işıl Karabey Aksakalli, Turgay Çelik, Ahmet Burak Can, and Bedir Tekinerdoğan. 2021. Deployment and communication patterns in microservice architectures: A systematic literature review. *Journal of Systems and Software* 180 (2021), 111014. DOI : <https://doi.org/10.1016/j.jss.2021.111014>

- [68] Ioanna Karageorgou, Panagiotis Liakos, and Alex Delis. 2020. A sentiment analysis service platform for streamed multilingual tweets. In *Proceedings of the 2020 IEEE International Conference on Big Data (Big Data)*. 3262–3271. DOI : <https://doi.org/10.1109/BigData50022.2020.9377837>
- [69] Narsimlu Kemsaram, Anwesha Das, and Gijs Dubbelman. 2020. Architecture design and development of an on-board stereo vision system for cooperative automated vehicles. In *Proceedings of the 2020 IEEE 23rd International Conference on Intelligent Transportation Systems (ITSC) (2020)*, 1–8.
- [70] Tom Killalea. 2016. The hidden dividends of microservices. *Communications of the ACM* 59, 8 (2016), 42–45. DOI : <https://doi.org/10.1145/2948985>
- [71] Barbara Kitchenham, O. Pearl Brereton, David Budgen, Mark Turner, John Bailey, and Stephen Linkman. 2009. Systematic literature reviews in software engineering—a systematic literature review. *Information and software technology* 51, 1 (2009), 7–15.
- [72] Barbara Kitchenham and Pearl Brereton. 2013. A systematic review of systematic review process research in software engineering. *Information and Software Technology* 55, 12 (2013), 2049–2075. DOI : <https://doi.org/10.1016/j.infsof.2013.07.010>
- [73] Barbara Ann Kitchenham and Stuart Charters. 2007. *Guidelines for Performing Systematic Literature Reviews in Software Engineering*. Technical Report EBSE 2007-001. Keele University and Durham University Joint Report. Retrieved from https://www.elsevier.com/_data/promis_misc/525444systematicreviewsguide.pdf
- [74] Martin Kleppmann, Adam Wiggins, Peter van Hardenberg, and Mark McGranaghan. 2019. Local-first software: You own your data, in spite of the cloud. In *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward! 2019)*. Association for Computing Machinery, New York, NY, USA, 154–178. DOI : <https://doi.org/10.1145/3359591.3359737>
- [75] Mohit Kumar, Kalka Dubey, and Rakesh Pandey. 2021. Evolution of emerging computing paradigm cloud to fog: Applications, limitations and research challenges. In *Proceedings of the 2021 11th International Conference on Cloud Computing, Data Science and Engineering (Confluence)*. 257–261. DOI : <https://doi.org/10.1109/Confluence51648.2021.9377050>
- [76] Matthew Lebofsky, Steve Croft, Andrew P. V. Siemion, Danny C. Price, J. Emilio Enriquez, Howard Isaacson, David H. E. MacMahon, David Anderson, Bryan Brzycki, Jeff Cobb, et al. 2019. The breakthrough listen search for intelligent life: Public data, formats, reduction, and archiving. *Publications of the Astronomical Society of the Pacific* 131, 1006 (2019), 124505. DOI : <https://doi.org/10.1088/1538-3873/ab3e82>
- [77] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. 2015. Deep learning. *Nature* 521, 7553 (2015), 436–444.
- [78] Grace A. Lewis and Dennis B. Smith. 2008. Service-oriented architecture and its implications for software maintenance and evolution. In *Proceedings of the 2008 Frontiers of Software Maintenance*. IEEE, 1–10.
- [79] Qiuchen Lu, Ajith Kumar Parlikad, Philip Woodall, Gishan Don Ranasinghe, Xiang Xie, Zhenglin Liang, Eirini Konstantinou, James Heaton, and Jennifer Schooling. 2020. Developing a digital twin at building and city levels: Case study of west cambridge campus. *Journal of Management in Engineering* 36, 3 (2020), 05020004. DOI : [https://doi.org/10.1061/\(ASCE\)ME.1943-5479.0000763](https://doi.org/10.1061/(ASCE)ME.1943-5479.0000763)
- [80] Lucy Ellen Lwakatare, Aiswarya Raj, Ivica Crnkovic, Jan Bosch, and Helena Holmström Olsson. 2020. Large-scale machine learning systems in real-world industrial settings: A review of challenges and solutions. *Information and Software Technology* 127 (2020), 106368. DOI : <https://doi.org/10.1016/j.infsof.2020.106368>
- [81] Patricia López Martínez, Ricardo Dintén, José María Drake, and Marta Zorrilla. 2021. A big data-centric architecture metamodel for Industry 4.0. *Future Generation Computer Systems* 125 (2021), 263–284. DOI : <https://doi.org/10.1016/j.future.2021.06.020>
- [82] Fang Miao, Wenjie Fan, Wenhui Yang, and Yan Xie. 2019. The study of data-oriented and ownership-based security architecture in open internet environment. In *Proceedings of the 3rd International Conference on Cryptography, Security and Privacy*. 121–129.
- [83] Kesari Mishra and Kishor S. Trivedi. 2011. Uncertainty propagation through software dependability models. In *Proceedings of the 2011 IEEE 22nd International Symposium on Software Reliability Engineering*. IEEE, 80–89.
- [84] Sylvie Delacroix, Joelle Pineau, and Jessica Montgomery. 2021. Democratising the Digital Revolution: The role of data governance. In *Reflections on Artificial Intelligence for Humanity*, Bertrand Braunschweig and Malik Ghallab (Eds.). Springer International Publishing, Cham, 40–52. DOI : https://doi.org/10.1007/978-3-030-69128-8_3
- [85] Henry Muccini and Karthik Vaidhyanathan. 2021. Software architecture for ML-based Systems: What exists and what lies ahead. In *Proceedings of the 2021 IEEE/ACM 1st Workshop on AI Engineering—Software Engineering for AI (WAIN)*. 121–128. DOI : <https://doi.org/10.1109/WAIN52551.2021.00026>
- [86] Raja Mubashir Munaf, Jawwad Ahmed, Faraz Khakwani, and Tauseef Rana. 2019. Microservices architecture: Challenges and proposed conceptual design. In *Proceedings of the 2019 International Conference on Communication Technologies (ComTech)*. 82–87. DOI : <https://doi.org/10.1109/COMTECH.2019.8737831>

- [87] Martin M. Müller and Marcel Salathé. 2019. Crowdbreaks: Tracking health trends using public social media data and crowdsourcing. *Frontiers in Public Health* Volume 7-2019, (2019). DOI : <https://doi.org/10.3389/fpubh.2019.00081>
- [88] Alfredo Nazabal, Christopher K. I. Williams, Giovanni Colavizza, Camila Rangel Smith, and Angus Williams. 2020. Data engineering for data analytics: A classification of the issues, and case studies. *arXiv preprint arXiv:2004.12929*. Retrieved from <http://arxiv.org/abs/2004.12929>
- [89] Anh Tuan Nguyen, Markus W. Drealan, Diu Khue Luu, Ming Jiang, Jian Xu, Jonathan Cheng, Qi Zhao, Edward W. Keefer, and Zhi Yang. 2021. A portable, self-contained neuroprosthetic hand with deep learning-based finger control. *Journal of Neural Engineering* 18, 5 (2021), 056051. DOI : <https://doi.org/10.1088/1741-2552/ac2a8d>
- [90] Stoyan Nikolov. 2018. OOP is dead, long live data-oriented design. CppCon. Retrieved February 2, 2019 from <https://www.youtube.com/watch>
- [91] Long Niu, Sachio Saiki, and Masahide Nakamura. 2017. Recognizing ADLs of one person household based on non-intrusive environmental sensing. In *Proceedings of the 2017 18th IEEE/ACIS International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD)*. 477–482. DOI : <https://doi.org/10.1109/SNPD.2017.8022765>
- [92] A. Jefferson Offutt, Mary Jean Harrold, and Priyadarshan Kolte. 1993. A software metric system for module coupling. *Journal of Systems and Software* 20, 3 (1993), 295–308.
- [93] Robert Olsson. 2014. *Applying REST Principles on Local Client-Side APIs*. Master's thesis. KTH, School of Computer Science and Communication (CSC).
- [94] OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, Red Avila, Igor Babuschkin, Suchir Balaji, Valerie Balcom, Paul Baltescu, Haiming Bao, Mohammad Bavarian, Jeff Belgum, Irwan Bello, Jake Berdine, Gabriel Bernadett-Shapiro, Christopher Berner, Lenny Bogdonoff, Oleg Boiko, Madelaine Boyd, Anna-Luisa Brakman, Greg Brockman, Tim Brooks, Miles Brundage, Kevin Button, Trevor Cai, Rosie Campbell, Andrew Cann, Brittany Carey, Chelsea Carlson, Rory Carmichael, Brooke Chan, Che Chang, Fotis Chantzis, Derek Chen, Sully Chen, Ruby Chen, Jason Chen, Mark Chen, Ben Chess, Chester Cho, Casey Chu, Hyung Won Chung, Dave Cummings, Jeremiah Currier, Yunxing Dai, Cory Decareaux, Thomas Degry, Noah Deutsch, Damien Deville, Arka Dhar, David Dohan, Steve Dowling, Sheila Dunning, Adrien Ecoffet, Atty Eleti, Tyna Eloundou, David Farhi, Liam Fedus, Niko Felix, Simón Posada Fishman, Juston Forte, Isabella Fulford, Leo Gao, Elie Georges, Christian Gibson, Vik Goel, Tarun Gogineni, Gabriel Goh, Rapha Gontijo-Lopes, Jonathan Gordon, Morgan Grafstein, Scott Gray, Ryan Greene, Joshua Gross, Shixiang Shane Gu, Yufei Guo, Chris Hallacy, Jesse Han, Jeff Harris, Yuchen He, Mike Heaton, Johannes Heidecke, Chris Hesse, Alan Hickey, Wade Hickey, Peter Hoeschele, Brandon Houghton, Kenny Hsu, Shengli Hu, Xin Hu, Joost Huizinga, Shantanu Jain, Shawn Jain, Joanne Jang, Angela Jiang, Roger Jiang, Haozhun Jin, Denny Jin, Shino Jomoto, Billie Jonn, Heewoo Jun, Tomer Kaftan, Lukasz Kaiser, Ali Kamali, Ingmar Kanitscheider, Nitish Shirish Keskar, Tabarak Khan, Logan Kilpatrick, Jong Wook Kim, Christina Kim, Yongjik Kim, Jan Hendrik Kirchner, Jamie Kiros, Matt Knight, Daniel Kokotajlo, Lukasz Kondraciuk, Andrew Kondrich, Aris Konstantinidis, Kyle Kopic, Gretchen Krueger, Vishal Kuo, Michael Lampe, Ikai Lan, Teddy Lee, Jan Leike, Jade Leung, Daniel Levy, Chak Ming Li, Rachel Lim, Molly Lin, Stephanie Lin, Mateusz Litwin, Theresa Lopez, Ryan Lowe, Patricia Lue, Anna Makanju, Kim Malfacini, Sam Manning, Todor Markov, Yaniv Markovski, Bianca Martin, Katie Mayer, Andrew Mayne, Bob McGrew, Scott Mayer McKinney, Christine McLeavey, Paul McMillan, Jake McNeil, David Medina, Aalok Mehta, Jacob Menick, Luke Metz, Andrey Mishchenko, Pamela Mishkin, Vinnie Monaco, Evan Morikawa, Daniel Mossing, Tong Mu, Mira Murati, Oleg Murk, David Mély, Ashvin Nair, Reiichiro Nakano, Rajeev Nayak, Arvind Neelakantan, Richard Ngo, Hyeonwoo Noh, Long Ouyang, Cullen O'Keefe, Jakub Pachocki, Alex Paino, Joe Palermo, Ashley Pantuliano, Giambattista Parascandolo, Joel Parish, Emy Parparita, Alex Passos, Mikhail Pavlov, Andrew Peng, Adam Perelman, Filipe de Avila Belbute Peres, Michael Petrov, Henrique Ponde de Oliveira Pinto, Michael, Pokorny, Michelle Pokrass, Vitchyr H. Pong, Tolly Powell, Alethea Power, Boris Power, Elizabeth Proehl, Raul Puri, Alec Radford, Jack Rae, Aditya Ramesh, Cameron Raymond, Francis Real, Kendra Rimbach, Carl Ross, Bob Rotsted, Henri Roussez, Nick Ryder, Mario Saltarelli, Ted Sanders, Shibani Santurkar, Girish Sastry, Heather Schmidt, David Schnurr, John Schulman, Daniel Selsam, Kyla Sheppard, Toki Sherbakov, Jessica Shieh, Sarah Shoker, Pranav Shyam, Szymon Sidor, Eric Sigler, Maddie Simens, Jordan Sitkin, Katarina Slama, Ian Sohl, Benjamin Sokolowsky, Yang Song, Natalie Staudacher, Felipe Petroski Such, Natalie Summers, Ilya Sutskever, Jie Tang, Nikolas Tezak, Madeleine B. Thompson, Phil Tillet, Amin Tootoonchian, Elizabeth Tseng, Preston Tuggle, Nick Turley, Jerry Tworek, Juan Felipe Cerón Uribe, Andrea Vallone, Arun Vijayvergiya, Chelsea Voss, Carroll Wainwright, Justin Jay Wang, Alvin Wang, Ben Wang, Jonathan Ward, Jason Wei, CJ Weinmann, Akila Welihinda, Peter Welinder, Jiayi Weng, Lilian Weng, Matt Wiethoff, Dave Willner, Clemens Winter, Samuel Wolrich, Hannah Wong, Lauren Workman, Sherwin Wu, Jeff Wu, Michael Wu, Kai Xiao, Tao Xu, Sarah Yoo, Kevin Yu, Qiming Yuan, Wojciech Zaremba, Rowan Zellers, Chong Zhang, Shengjia Zhao, Tianhao Zheng, Juntang Zhuang, William Zhuk, and Barret Zoph. 2024. GPT-4 Technical Report. *ArXiv abs/2303.08774*, (March 2024). Retrieved from <http://arxiv.org/abs/2303.08774>

- [95] Mike Loukides and Steve Swoyer. 2020. Microservices Adoption in 2020: A survey. O'Reilly. Retrieved from <https://www.oreilly.com/radar/microservices-adoption-in-2020/>
- [96] Genevieve B. Orr and Klaus-Robert Müller. 2003. *Neural Networks: Tricks of the Trade*. Springer.
- [97] Claus Pahl. 2023. Research challenges for machine learning-constructed software. *Service Oriented Computing and Applications* 17, 1 (2023), 1–4.
- [98] Tom Le Paine, Cosmin Paduraru, Andrea Michi, Caglar Gulcehre, Konrad Zolna, Alexander Novikov, Ziyu Wang, and Nando de Freitas. 2020. Hyperparameter selection for offline reinforcement learning. *ArXiv abs/2303.08774*. Retrieved from <http://arxiv.org/abs/2007.09055>
- [99] Andrei Paleyes, Christian Cabrera, and Neil D. Lawrence. 2022. An empirical evaluation of flow based programming in the machine learning deployment context. In *Proceedings of the 2022 IEEE/ACM 1st International Conference on AI Engineering—Software Engineering for AI (CAIN)*. 54–64.
- [100] Andrei Paleyes, Siyuan Guo, Bernhard Scholkopf, and Neil D. Lawrence. 2023. Dataflow graphs as complete causal graphs. In *Proceedings of the 2023 IEEE/ACM 2nd International Conference on AI Engineering—Software Engineering for AI (CAIN)*. 7–12. DOI : <https://doi.org/10.1109/CAIN58948.2023.00010>
- [101] Andrei Paleyes, Siyuan Guo, Bernhard Scholkopf, and Neil D. Lawrence. 2023. Dataflow graphs as complete causal graphs. In *Proceedings of the 2023 IEEE/ACM 2nd International Conference on AI Engineering—Software Engineering for AI (CAIN)*.
- [102] Andrei Paleyes and Neil David Lawrence. 2023. Causal fault localisation in dataflow systems. In *Proceedings of the 3rd Workshop on Machine Learning and Systems (EuroMLSys '23)*. Association for Computing Machinery, New York, NY, USA, 140–147. DOI : <https://doi.org/10.1145/3578356.3592593>
- [103] Andrei Paleyes, Henry B. Moss, Victor Picheny, Piotr Zulawski, and Felix Newman. 2022. A penalisation method for batch multi-objective Bayesian optimisation with application in heat exchanger design. In *Proceedings of the Workshop on Adaptive Experimental Design and Active Learning in the Real World, ICML*. DOI : <https://doi.org/10.48550/ARXIV.2206.13326>
- [104] Andrei Paleyes, Mark Pullin, Maren Mahsereci, Neil Lawrence, and Javier González. 2019. Emulation of physical processes with Emukit. In *Proceedings of the 2nd Workshop on Machine Learning and the Physical Sciences, NeurIPS*.
- [105] Andrei Paleyes, Raoul-Gabriel Urma, and Neil D. Lawrence. 2022. Challenges in deploying machine learning: A survey of case studies. *ACM Comput. Surv.* 55, 6 (December 2022). DOI : <https://doi.org/10.1145/3533378>
- [106] Milan Patel, Brian Naughton, Caroline Chan, Nurit Sprecher, Sadayuki Abeta, Adrian Neal, and others. 2014. Mobile-edge computing introductory technical white paper. 1089–7801.
- [107] Jan Pennekamp, Martin Henze, Simo Schmidt, Philipp Niemietz, Marcel Fey, Daniel Trauth, Thomas Bergs, Christian Brecher, and Klaus Wehrle. 2019. Dataflow challenges in an internet of production: A security and privacy perspective. In *Proceedings of the ACM Workshop on Cyber-Physical Systems Security and Privacy*. 27–38.
- [108] Neoklis Polyzotis, Sudip Roy, Steven Euijong Whang, and Martin Zinkevich. 2018. Data lifecycle challenges in production machine learning: A survey. *ACM SIGMOD Record* 47, 2 (2018), 17–28.
- [109] S. Joe Qin. 2012. Survey on data-driven industrial process monitoring and diagnosis. *Annual Reviews in Control* 36, 2 (2012), 220–234.
- [110] Xiwei Qiu, Yuanshun Dai, Peng Sun, and Xin Jin. 2020. PHM technology for memory anomalies in cloud computing for IaaS. In *Proceedings of the 2020 IEEE 20th International Conference on Software Quality, Reliability and Security (QRS)*. 41–51. DOI : <https://doi.org/10.1109/QRS51102.2020.00018>
- [111] Luis Quintero, Panagiotis Papapetrou, John E. Muñoz, and Uno Fors. 2019. Implementation of mobile-based real-time heart rate variability detection for personalized healthcare. In *Proceedings of the 2019 International Conference on Data Mining Workshops (ICDMW)*. 838–846. DOI : <https://doi.org/10.1109/ICDMW.2019.00123>
- [112] Mohaimenul Azam Khan Raiaan, Md. Saddam Hossain Mukta, Kaniz Fatema, Nur Mohammad Fahad, Sadman Sakib, Most Marufatul Jannat Mim, Jubaer Ahmad, Mohammed Eunus Ali, and Sami Azam. 2024. A review on large language models: architectures, applications, taxonomies, open issues and challenges. *IEEE Access* 12 (2024), 26839–26874. DOI : <https://doi.org/10.1109/ACCESS.2024.3365742>
- [113] Diana Robinson, Christian Cabrera, Andrew D. Gordon, Neil D. Lawrence, and Lars Mennen. 2025. Requirements are all you need: The final frontier for end-user software engineering. *ACM Trans. Softw. Eng. Methodol.* 34, 5 (May 2025). DOI : <https://doi.org/10.1145/3708524>
- [114] Marouane Salhaoui, Antonio Guerrero Gonzalez, Mounir Arioua, Juan Carlos Molina Molina, Francisco J. Ortiz, and Ahmed El Oualkadi. 2020. Edge-cloud architectures using UAVs dedicated to industrial IoT monitoring and control applications. In *Proceedings of the 2020 International Symposium on Advanced Electrical and Communication Technologies (ISAECT)*. 1–6. DOI : <https://doi.org/10.1109/ISAECT50560.2020.9523700>
- [115] Luis Sanchez, Luis Muñoz, Jose Antonio Galache, Pablo Sotres, Juan R. Santana, Veronica Gutierrez, Rajiv Ramdhany, Alex Gluhak, Srdjan Krco, Evangelos Theodoridis, and Dennis Pfisterer. 2014. SmartSantander: IoT experimentation over a smart city testbed. *Computer Networks* 61 (2014), 217–238. DOI : <https://doi.org/10.1016/j.bjp.2013.12.020>

- [116] Juan Ramón Santana, Luis Sánchez, Pablo Sotres, Jorge Lanza, Tomás Llorente, and Luis Muñoz. 2020. A privacy-aware crowd management system for smart cities and smart buildings. *IEEE Access* 8 (2020), 135394–135405. DOI : <https://doi.org/10.1109/ACCESS.2020.3010609>
- [117] David Sarabia-Jácome, Regel Usach, Carlos E. Palau, and Manuel Esteve. 2020. Highly-efficient fog-based deep learning AAL fall detection system. *Internet of Things* 11 (2020), 100185. DOI : <https://doi.org/10.1016/j.iot.2020.100185>
- [118] Tim Schopf, Daniel Braun, and Florian Matthes. 2021. Lbl2Vec: An embedding-based approach for unsupervised document retrieval on predefined topics. In *Proceedings of the 17th International Conference on Web Information Systems and Technologies—WEBIST*. 124–132. DOI : <https://doi.org/10.5220/0010710300003058>
- [119] Carson Schubert, Kevin Black, Daniel Fonseca, Abhimanyu Dhir, Jacob Deutsch, Nihal Dhamani, Gavin Martin, and Maruthi Akella. 2021. A pipeline for vision-based on-orbit proximity operations using deep learning and synthetic imagery. In *Proceedings of the 2021 IEEE Aerospace Conference*. 1–15. DOI : <https://doi.org/10.1109/AERO50100.2021.9438232>
- [120] Robert Schuler, Carl Kesselman, and Karl Czajkowski. 2015. Data centric discovery with a data-oriented architecture. In *Proceedings of the 1st Workshop on the Science of Cyberinfrastructure: Research, Experience, Applications and Models (SCREAM '15)*. Association for Computing Machinery, New York, NY, USA, 37–44. DOI : <https://doi.org/10.1145/2753524.2753532>
- [121] Johann Schumann, Timmy Mbaya, and Ole J. Mengshoel. 2012. Software and system health management for autonomous robotics missions. Carnegie Mellon University (September 2012). DOI : <https://doi.org/10.1184/R1/6710654.v1>
- [122] Roy Schwartz, Jesse Dodge, Noah A. Smith, and Oren Etzioni. 2020. Green AI. *Communications of the ACM* 63, 12 (2020), 54–63. DOI : <https://doi.org/10.1145/3381831>
- [123] Malte Schwarzkopf, Eddie Kohler, M. Frans Kaashoek, and Robert Tappan Morris. 2019. Position: GDPR compliance by construction. In *Proceedings of the Poly/DMAH@VLDB*.
- [124] Guohou Shan, Boxin Zhao, James R. Clavin, Haibin Zhang, and Sisi Duan. 2022. Poligraph: Intrusion-tolerant and distributed fake news detection system. *IEEE Transactions on Information Forensics and Security* 17 (2022), 28–41. DOI : <https://doi.org/10.1109/TIFS.2021.3131026>
- [125] Weisong Shi, Jie Cao, Quan Zhang, Youhuizi Li, and Lanyu Xu. 2016. Edge computing: Vision and challenges. *IEEE Internet of Things Journal* 3, 5 (2016), 637–646.
- [126] Wanxin Shi, Qing Li, Chao Wang, Gengbiao Shen, Weichao Li, Yu Wu, and Yong Jiang. 2019. LEAP: Learning-based smart edge with caching and prefetching for adaptive video streaming. In *Proceedings of the 2019 IEEE/ACM 27th International Symposium on Quality of Service (IWQoS)*. 1–10. DOI : <https://doi.org/10.1145/3326285.3329051>
- [127] Chihhsiong Shih, Lai Youchen, Cheng-hsu Chen, and William Cheng-Chung Chu. 2020. An early warning system for hemodialysis complications utilizing transfer learning from HD IoT dataset. In *Proceedings of the 2020 IEEE 44th Annual Computers, Software, and Applications Conference (COMPSAC)*. 759–767. DOI : <https://doi.org/10.1109/COMPSAC48688.2020.0-168>
- [128] Jatinder Singh, Jennifer Cobbe, and Chris Norval. 2018. Decision provenance: Harnessing data flow for accountable systems. *IEEE Access* 7 (2018), 6562–6574.
- [129] Neha Singh and Kirti Tyagi. 2015. A Literature Review of the Reliability of Composite Web Service in Service-Oriented Architecture. *SIGSOFT Software Engineering Notes* 40, 1 (2015), 1–8. DOI : <https://doi.org/10.1145/2693208.2693237>
- [130] Ben Stopford. 2016. The Data Dichotomy: Rethinking the Way We Treat Data and Services. Confluent. Retrieved from <https://www.confluent.io/blog/data-dichotomy-rethinking-the-way-we-treat-data-and-services/>
- [131] Shabnam Sultana, Philippe Dooze, and Vijay Venkatesh Kumar. 2021. Realization of an intrusion detection use-case in ONAP with acumos. In *Proceedings of the 2021 International Conference on Computer Communications and Networks (ICCCN)*. 1–6. DOI : <https://doi.org/10.1109/ICCCN52240.2021.9522281>
- [132] Hadi Tabatabaee Malazi, Saqib Rasool Chaudhry, Aqeel Kazmi, Andrei Palade, Christian Cabrera, Gary White, and Siobhán Clarke. 2022. Dynamic service placement in multi-access edge computing: A systematic literature review. *IEEE Access* 10 (2022), 32639–32688. DOI : <https://doi.org/10.1109/ACCESS.2022.3160738>
- [133] Davide Taibi, Valentina Lenarduzzi, and Claus Pahl. 2018. Architectural patterns for microservices: A systematic mapping study. In *CLOSER 2018—Proceedings of the 8th International Conference on Cloud Computing and Services Science*. SCITEPRESS, 221–232. DOI : <https://doi.org/10.5220/0006798302210232>
- [134] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*. Retrieved from <http://arxiv.org/abs/2302.13971>
- [135] Lillian Tsai, Malte Schwarzkopf, and Eddie Kohler. 2021. Privacy heroes need data disguises. In *Proceedings of the Workshop on Hot Topics in Operating Systems*. 112–118.
- [136] Lorenzo Vaccaro, Giuseppe Sansonetti, and Alessandro Micarelli. 2021. An empirical review of automated machine learning. *Computers* 10, 1 (2021), 11.

- [137] Richard Van Noorden and Jeffrey M. Perkel. 2023. AI and science: what 1,600 researchers think. *Nature* 621, 7980 (2023), 672–675.
- [138] Christian Vorhemus and Erich Schikuta. 2017. A data-oriented architecture for loosely coupled real-time information systems. In *Proceedings of the 19th International Conference on Information Integration and Web-based Applications and Services*. 472–481.
- [139] Jonathan Waring, Charlotta Lindvall, and Renato Umeton. 2020. Automated machine learning: Review of the state-of-the-art and opportunities for healthcare. *Artificial Intelligence in Medicine* 104 (2020), 101822. DOI: <https://doi.org/10.1016/j.artmed.2020.101822>
- [140] Hironori Washizaki, Foutse Khomh, Yann-Gael Gueheneuc, Hironori Takeuchi, Naotake Natori, Takuo Doi, and Satoshi Okuda. 2022. Software-engineering design patterns for machine learning applications. *Computer* 55, 3 (March 2022), 30–39. DOI: <https://doi.org/10.1109/MC.2021.3137227>
- [141] Dawei Wei, Huansheng Ning, Feifei Shi, Yueliang Wan, Jiabo Xu, Shunkun Yang, and Li Zhu. 2021. Dataflow management in the internet of things: Sensing, control, and security. *Tsinghua Science and Technology* 26, 6 (2021), 918–930.
- [142] Danny Weyns. 2019. Software Engineering of Self-adaptive Systems. In *Handbook of Software Engineering*, Sungdeok Cha, Richard N. Taylor, and Kyocho Kang (Eds.). Springer International Publishing, Cham, 399–443. DOI: https://doi.org/10.1007/978-3-030-00262-6_11
- [143] Lauren J. Wong, I. V. William H. Clark, Bryse Flowers, R. Michael Buehrer, Alan J. Michaels, and William C. Headley. 2020. The RFML ecosystem: A look at the unique challenges of applying deep learning to radio frequency applications. *arXiv*. Retrieved from <https://arxiv.org/abs/2010.00432>
- [144] Hailu Xu, Boyuan Guan, Pinchao Liu, William Escudero, and Liting Hu. 2018. Harnessing the nature of spam in scalable online social spam detection. In *Proceedings of the 2018 IEEE International Conference on Big Data (Big Data)*. 3733–3736. DOI: <https://doi.org/10.1109/BigData.2018.8621926>
- [145] Deze Zeng, Lin Gu, Song Guo, Zixue Cheng, and Shui Yu. 2016. Joint optimization of task scheduling and image placement in fog computing supported software-defined embedded system. *IEEE Transactions on Computers* 65, 12 (2016), 3702–3712. DOI: <https://doi.org/10.1109/TC.2016.2536019>
- [146] Leihan Zhang, Jichang Zhao, and Ke Xu. 2016. Emotion-based social computing platform for streaming big-data: Architecture and application. In *Proceedings of the 2016 13th International Conference on Service Systems and Service Management (ICSSSM)*. 1–6. DOI: <https://doi.org/10.1109/ICSSSM.2016.7538620>
- [147] Tao Zhang, Biyun Ding, Xin Zhao, Ganjun Liu, and Zhibo Pang. 2021. LearningADD: Machine learning based acoustic defect detection in factory automation. *Journal of Manufacturing Systems* 60 (2021), 48–58. DOI: <https://doi.org/10.1016/j.jmsy.2021.04.005>
- [148] Jonathan Zittrain. 2022. *Intellectual Debt: With Great Power Comes Great Ignorance*. Cambridge University Press.

Received 9 October 2023; revised 1 September 2025; accepted 15 September 2025